

FIRST EDITION

DESIGN PATTERNS IN UNITY 6

Build Scalable and Maintainable
Games with C#



Daniel Buckley
Pablo Farias



Design Patterns in Unity 6: Build Scalable and Maintainable Games with C#

Zenva Academy

© 2026 Zenva Pty Ltd. All rights reserved

[OceanofPDF.com](https://oceanofpdf.com)

Table of Contents

[Introduction](#)

[How to Use This Book](#)

[A Quick Note Before We Begin](#)

[Getting Started](#)

[Prerequisites](#)

[Project Files](#)

[Recommended Unity Version](#)

[Installing Unity 6.3 LTS](#)

[Coding Patterns in Unity and C#](#)

[The SOLID Principles](#)

[Patterns Covered in This Book](#)

[SOLID Principles](#)

[What are the SOLID Principles?](#)

[Single Responsibility Principle](#)

[Open-Closed Principle](#)

[Liskov Substitution Principle](#)

[Interface Segregation Principle](#)

[Dependency Inversion Principle](#)

[What Is Coupling?](#)

[Events](#)

[Interfaces](#)

[Summary](#)

[Singleton Pattern](#)

[Why Use a Singleton?](#)

[Implementing a Basic Singleton](#)

[Adding the Instance Property](#)

[Setting the Instance in Awake](#)

[Preventing Duplicate Instances](#)

[Testing the Singleton](#)

[Accessing the Singleton from Other Scripts](#)

[When Singletons Are Useful](#)

[Creating a Generic Singleton Base Class](#)

[Adding a Generic Type Parameter](#)

[Writing the Awake Method](#)

[Using the Generic Singleton in GameManager](#)

[Reusing the Pattern for Other Managers](#)

[Extending the Base Class with DontDestroyOnLoad](#)

[Summary](#)

Observer Pattern

The Initial Setup

What the Observer Pattern Solves

Refactoring LightManager with Events

Testing the Events

Adding a Singleton to LightManager

Subscribing to Events in LightController

Fixing the Script Execution Order

The Result: Scalable Light Control

Summary

Command Pattern

Why Use the Command Pattern?

The Example Scene

Scene Setup and Scripts

Defining the ICommand Interface

Creating the MoveCommand Class

Creating the PlayerMovement Script

Building the CommandExecutor

Handling Player Input

Executing Commands and Recording History.

[Testing Basic Movement](#)

[Implementing Undo](#)

[Implementing Redo](#)

[Connecting the UI Buttons](#)

[Practical Uses of the Command Pattern](#)

[Extending the System with a New Command](#)

[Creating the ScaleCommand Script](#)

[Adding Scale Support to PlayerMovement](#)

[Completing the ScaleCommand Class](#)

[Triggering ScaleCommand from Input](#)

[Testing the Extended System](#)

[Summary](#)

[Object Pooling](#)

[Why Object Pooling Matters](#)

[Building a Reusable ObjectPool Script](#)

[Creating New Pooled Objects](#)

[Pre-Spawning the Pool in Start](#)

[Retrieving Objects from the Pool](#)

[Adding the Pool to the Scene](#)

[Refactoring Shooter to Use the Pool](#)

[Returning Projectiles to the Pool](#)

[Wiring the Pool Reference in the Inspector](#)

[Testing the Pooling System](#)

[Performance Benefits of Object Pooling](#)

[Summary](#)

[Closing Words](#)

[What You Accomplished](#)

[Where to Go From Here](#)

[Final Thought](#)

[Full Source Code](#)

[Singleton Pattern](#)

[GameManager.cs](#)

[Singleton.cs](#)

[TestScript.cs](#)

[Observer Pattern](#)

[LightController.cs](#)

[LightManager.cs](#)

[Command Pattern](#)

[CommandExecutor.cs](#)

[ICommand.cs](#)

[MoveCommand.cs](#)

[PlayerMovement.cs](#)

[ScaleCommand.cs](#)

[Object Pooling](#)

[ObjectPool.cs](#)

[Projectile.cs](#)

[Shooter.cs](#)

[*OceanofPDF.com*](#)

Introduction

Welcome, and thanks for picking up this book.

Game development is full of exciting ideas, but it can also get messy fast. A feature works, then another feature depends on it, and before long even small changes feel bigger than they should. This book is here to help you build with a little more clarity and confidence, so your Unity projects stay easier to understand, extend, and maintain as they grow.

Across several chapters and multiple hands-on projects, we will focus on writing code that feels more intentional. The goal is not just to make things work, but to make them work in a way that is easier to revisit later, easier to scale, and easier to reason about when your project becomes more complex.

How to Use This Book

This section will help you get the most value from the chapters ahead. The ideas in this book become much clearer when you actively apply them, rather than only reading through them.

Follow along as you read.

Try writing the code yourself, even if the example seems straightforward. Understanding builds through repetition, and small details tend to stick better when you type them out and see the results in Unity.

Experiment as you go.

Change variable names, tweak behaviors, and test what happens when you break something on purpose. That kind of curiosity is often where the real learning happens, because it helps you see not just what works, but why it works.

Pause when a concept clicks.

If you reach a point where an idea suddenly makes sense, take a moment to reflect on where else you could use it. A short pause here can make the difference between recognizing a pattern in a tutorial and being able to use it in your own game.

Use the appendix when you need a quick reference.

If you want to refresh your memory on a concept without rereading a full chapter, the appendix can help you find your footing quickly. Think of it as a practical companion for review, revision, and later use in your own projects.

A Quick Note Before We Begin

You do not need to master every idea on the first pass. Some concepts become much more natural after you have seen them a few times in different contexts. If something feels abstract at first, keep going, build it, and come back to it later. That is a normal part of learning to write better game code.

In the next chapter, we cover prerequisites, project files, and Unity installation so you have everything in place before diving into the patterns.

OceanofPDF.com

Getting Started

This chapter establishes the foundation for the rest of the book. We will clarify the expected background knowledge, point you to the project files, and make sure your Unity installation is ready before moving into the coding patterns themselves.

Prerequisites

This book assumes you already have intermediate familiarity with Unity and C# scripting. In particular, you should be comfortable with the following:

- C# fundamentals: variables, functions, classes, and access modifiers (public, private, static)
- Object-oriented programming: inheritance, abstract classes, interfaces, and a basic understanding of generics
- Unity's component system: attaching scripts to GameObjects, using [SerializeField], and understanding the MonoBehaviour lifecycle (Awake, Start, Update)
- Unity Editor navigation: Hierarchy, Inspector, Project window, and entering Play mode
- Basic scene skills: creating GameObjects, assigning components and materials, and organizing a simple scene

If you need to strengthen those fundamentals first, the following resources are recommended:

- [Game Development with Unity: Learn the Foundations of Creating Games with Unity](https://academy.zenva.com/product/intro-to-unity-ebook/) (https://academy.zenva.com/product/intro-to-unity-ebook/)

- [C# for Unity Developers: Learn Programming Fundamentals Through Mini Projects](https://academy.zenva.com/product/unity-mini-projects-ebook/) (https://academy.zenva.com/product/unity-mini-projects-ebook/)

Alternatively, you can work through the course versions:

- [Intro to Game Development with Unity](https://academy.zenva.com/product/intro-to-game-development-with-unity/) (https://academy.zenva.com/product/intro-to-game-development-with-unity/)
- [Unity Mini-Projects – C# Fundamentals](https://academy.zenva.com/product/minigame-projects-explore-game-development-c/) (https://academy.zenva.com/product/minigame-projects-explore-game-development-c/)

You may also benefit from additional beginner-level Unity study before continuing. We recommend having one to three Unity projects under your belt before jumping in. Even so, this book will guide you step by step as we build and discuss each concept.

Project Files

To follow along with the examples in this book, download the completed project for the supported Unity version below:

- [Starter Project](https://academy.zenva.com/wp-content/uploads/2026/05/Starter-Project-Unity-Coding-Patterns-v6.3.zip) (https://academy.zenva.com/wp-content/uploads/2026/05/Starter-Project-Unity-Coding-Patterns-v6.3.zip)
- [Complete Project](https://academy.zenva.com/wp-content/uploads/2026/05/Complete-Project-Unity-Coding-Patterns-v6.3.zip) (https://academy.zenva.com/wp-content/uploads/2026/05/Complete-Project-Unity-Coding-Patterns-v6.3.zip)

Zenva-Created Assets

All artwork and media assets included in the project files were created by **Zenva** and are released under:

Creative Commons CC0 (Public Domain)

<https://creativecommons.org/publicdomain/zero/1.0/>

Zenva-created source code included in the project files for this course is released under the MIT License (<https://opensource.org/license/MIT>). Third-party libraries and dependencies remain under their respective licenses.

Recommended Unity Version

Unity changes quickly, and small version differences can affect menus, workflows, and package behavior. For that reason, this book recommends using **Unity 6.3 LTS**.

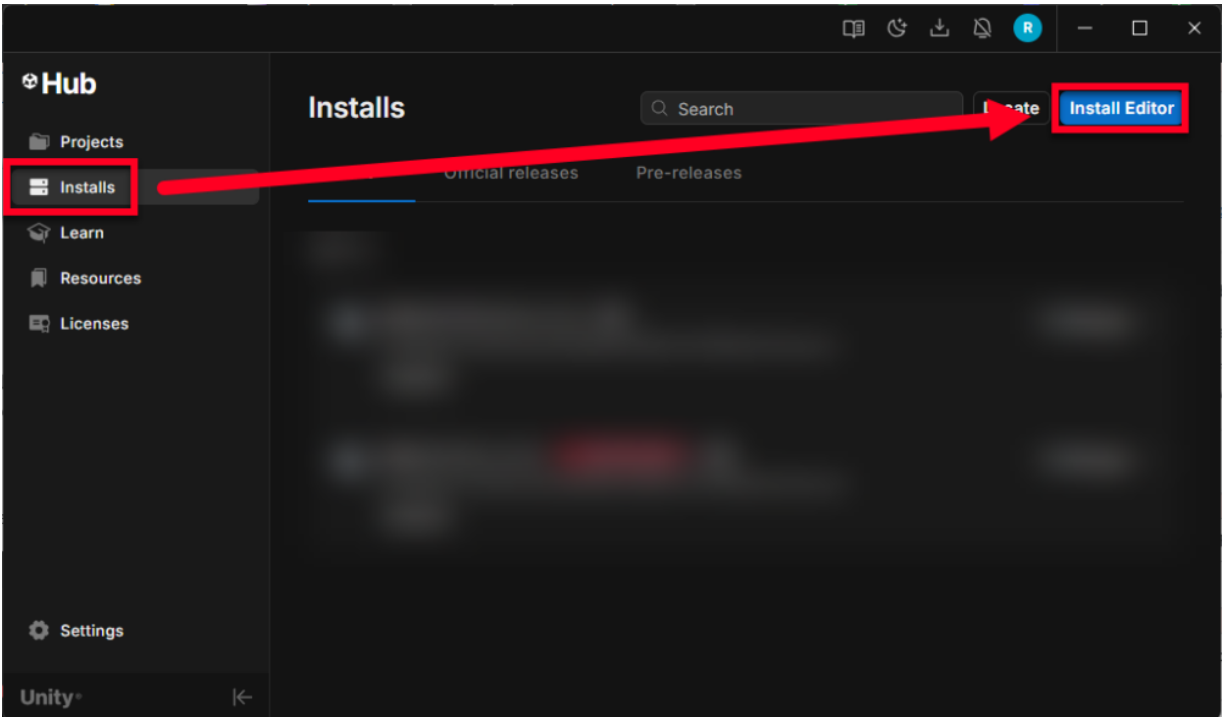
LTS stands for Long-Term Support. These releases are intended to provide a more stable foundation for learning and development, which makes them a strong choice for educational projects. You can learn more about Unity's LTS releases here: [Unity LTS releases](https://unity3d.com/unity/qa/lts-releases) (<https://unity3d.com/unity/qa/lts-releases>).

It is best not to use a newer version than the one recommended here, since some project files or steps may behave differently.

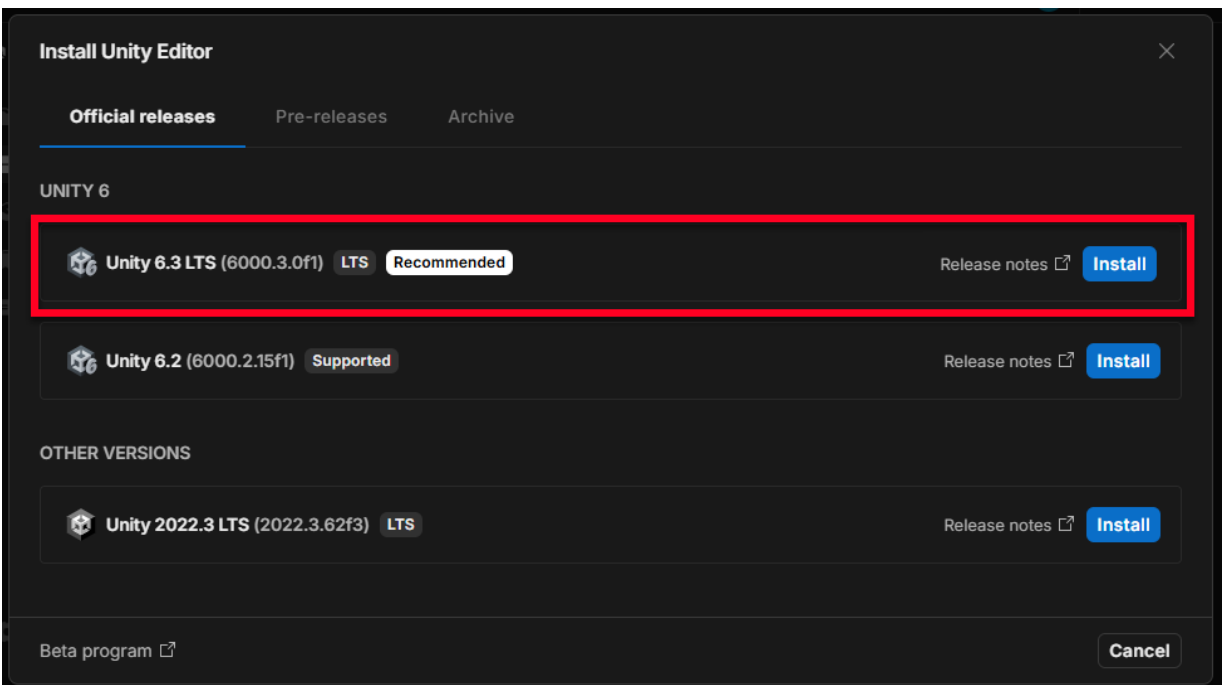
Installing Unity 6.3 LTS

If Unity is not installed yet, the easiest way to get started is through Unity Hub. This lets you manage editor versions and keep your setup aligned with the book.

To do so, open **Unity Hub** and go to the **Installs** section. From there, click the blue **Install Editor** button, as shown in the image below.



A new window will open with available Unity versions. In the **Official releases** list, look under the **Unity 6** section and select **Unity 6.3 [LTS]**.



Click **Install**, then follow the prompts to complete the setup.

Importantly, make sure you install [Visual Studio Community](https://visualstudio.microsoft.com/vs/community/) (<https://visualstudio.microsoft.com/vs/community/>) along with Unity. This is usually bundled with Unity, but make sure to not uncheck it.

Coding Patterns in Unity and C#

This book focuses on coding patterns in Unity using C#. The goal is to help you write code that is cleaner, less tightly coupled, and easier to maintain as your projects grow.

Coding patterns are reusable approaches to common programming problems. As a game project becomes more complex, code can quickly become difficult to manage if systems depend too heavily on one another or if small changes ripple through unrelated parts of the project. Patterns help reduce that friction by giving you proven structures for organizing your code.

These ideas are useful in projects of any size, from small prototypes to larger games. Once understood and applied well, they can make your codebase more modular and much easier to work with.

The SOLID Principles

Before looking at specific patterns, we will begin with the SOLID principles. These are five widely used guidelines for writing flexible, understandable object-oriented code.

Unlike design patterns, which often come with recognizable implementation structures, SOLID is more theoretical. It provides a set of principles you can use to evaluate how responsibilities are divided, how systems depend on one another, and how easy your code is to extend without breaking existing behavior.

This is especially valuable in team environments, where readable and predictable code matters just as much as functionality.

Patterns Covered in This Book

After the SOLID principles, the book explores four practical coding patterns that are especially relevant to Unity game development:

- The Singleton Pattern
- The Observer Pattern
- The Command Pattern
- Object Pooling

Each pattern will be introduced in context and applied through Unity-focused examples. Because these patterns map well to common gameplay systems, they are particularly useful tools for C# developers working in the engine.

Once everything is installed, you will be ready to begin exploring the principles and patterns used throughout the rest of this book.

Let's jump into our next chapter where we'll begin delving further into what the SOLID Principles are.

OceanofPDF.com

SOLID Principles

In the previous chapter, we introduced the SOLID principles as a foundation for writing cleaner, more maintainable Unity code. In this chapter, we will look at each principle in turn and connect it to practical C# scripting decisions you are likely to make in real projects.

What are the SOLID Principles?

SOLID is a set of five object-oriented design principles that help keep code flexible, readable, and easier to extend over time. These ideas are especially useful in Unity, where gameplay systems often grow quickly and can become difficult to manage if responsibilities are not clearly separated.

The five SOLID principles are:

- Single Responsibility Principle (SRP)
- Open-Closed Principle (OCP)
- Liskov Substitution Principle (LSP)
- Interface Segregation Principle (ISP)
- Dependency Inversion Principle (DIP)

We will begin with the first four principles, then finish with dependency inversion and the related idea of coupling.

Single Responsibility Principle

The Single Responsibility Principle states that each class or script should have one clear purpose. This matters because scripts that try to do too many things become harder to read, harder to test, and much harder to change safely.

In Unity, this often appears when a single script starts accumulating unrelated responsibilities. For example, a player script might begin by handling movement, then later gain score tracking, animation logic, UI updates, and input handling. At that point, even a small change can affect several systems at once.

A better approach is to divide those concerns into focused scripts. Instead of one large player script, you might use:

- **PlayerController** for movement
- **PlayerScoring** for score-related logic
- **PlayerAnimation** for animation behavior

This makes each script easier to understand and gives you more freedom to update one system without disturbing the others.

A common example of an SRP violation is a **GameManager** script that loads levels, manages score, and spawns enemies. These are separate concerns, so combining them into one class creates unnecessary complexity.

GameManager.cs

- Loads in the levels
- Manages player score
- Spawns in enemies

A cleaner design would split that work into dedicated scripts such as:

- **LevelLoader** for scene loading
- **ScoreManager** for score tracking
- **EnemySpawn** for enemy spawning

These scripts can still communicate when necessary, but each one now has a single, well-defined role.

Unity's component system already encourages this way of thinking. A **Rigidbody** handles physics. A **MeshRenderer** handles rendering. A **Collider** defines collision boundaries. Each component exists for a specific purpose, which is one reason Unity projects scale more cleanly when you follow the same pattern in your own scripts.

Unity Insight: Unity's component model and SRP

Unity builds every GameObject from small, focused components that each handle one concern. A Rigidbody drives physics. A MeshRenderer controls what the player sees. An AudioSource plays sound. None of these components knows what the others do, which is what makes them easy to add, remove, or swap independently.

Your own MonoBehaviour scripts become components too. Writing each script with a single, clear purpose keeps your code as composable as Unity's built-in components, and for the same reason.

Learn more: [Introduction to components](https://docs.unity3d.com/6000.3/Documentation/Manual/Components.html)
(<https://docs.unity3d.com/6000.3/Documentation/Manual/Components.html>)

Open-Closed Principle

The Open-Closed Principle states that classes should be open for extension but closed for modification. In practice, this means you should be able to add new behavior without repeatedly editing old, working code.

This is valuable because modifying existing classes over and over increases the chance of introducing bugs. If a class is designed well, new features can be added by building on top of it rather than rewriting it.

Consider an enemy class that contains separate methods for melee and ranged attacks. That may seem convenient at first, but it creates a problem: every time you add a new attack type, you must return to the original class and modify it. It also means the class contains behavior that not every enemy actually needs.

A better solution is to define a base enemy class with an abstract **Attack()** method, then create specialized enemy types that implement their own version of that attack.

```
public abstract class Enemy
{
    public abstract void Attack();
}
```

```
public class MeleeEnemy : Enemy
{
    public override void Attack ()
    {
        // Swing sword
    }
}

public class RangedEnemy : Enemy
{
    public override void Attack ()
    {
        // Shoot arrow
    }
}
```

With this design, adding a new enemy type such as a magic enemy does not require changing the original base class. You simply create another derived class and override **Attack()** with the new behavior.

The key idea is stability. The base class defines the shared contract, while new functionality is introduced through extension. This keeps existing code safer and makes future additions more predictable.

Liskov Substitution Principle

The Liskov Substitution Principle states that derived classes must be usable anywhere their base class is expected. In other words, if a system works with the base type, it should also work correctly with any subclass.

This principle helps ensure that inheritance is being used meaningfully. If a subclass cannot behave like the base class in practice, then the inheritance relationship is probably a poor fit.

For example, imagine an abstract **Weapon** class with subclasses such as **Sword** and **Bow**. If your player can equip and use a **Weapon**, then both **Sword** and **Bow** should work through that same shared interface or base behavior. The code should not need special exceptions just because the specific weapon type changes.

A useful Unity example is environmental damage. Suppose spikes in your level can damage any character that touches them. Rather than writing that logic only for the **Player** class, you could place shared damage behavior in a base **Character** class. Then both players and enemies can respond to spike damage in the same way.

This gives you a more flexible system. The spikes do not need to know whether they hit a player, an enemy, or some future character type. They only need to know they are interacting with a **Character**. That is the practical benefit of LSP: systems remain general, and subclasses fit naturally into them.

Interface Segregation Principle

The Interface Segregation Principle states that no class should be forced to depend on methods it does not use. You can think of this as the Single Responsibility Principle applied to interfaces.

This matters because large, general-purpose interfaces often create awkward implementations. Classes end up containing empty methods or placeholder behavior simply because the interface demands it, not because the class actually needs that functionality.

Consider an **IVehicle** interface with both **Drive()** and **Fly()** methods. A **Car** can drive, but it cannot fly. A **Helicopter** can fly, but it does not drive like a car. If both classes implement the same interface, each one is forced to define a method it does not meaningfully use.

The better approach is to split the interface into smaller, focused interfaces such as:

- **IGroundVehicle** with **Drive()**
- **IAirVehicle** with **Fly()**

Then a **Car** implements only **IGroundVehicle**, while a **Helicopter** implements only **IAirVehicle**.

This keeps interfaces more precise and makes implementing classes simpler. In Unity projects, this can be especially helpful when defining interactions between gameplay systems, because it allows scripts to depend only on the exact capabilities they need.

Dependency Inversion Principle

The final SOLID principle is the Dependency Inversion Principle. This principle focuses on reducing direct dependencies between systems, which helps keep your codebase modular and easier to change.

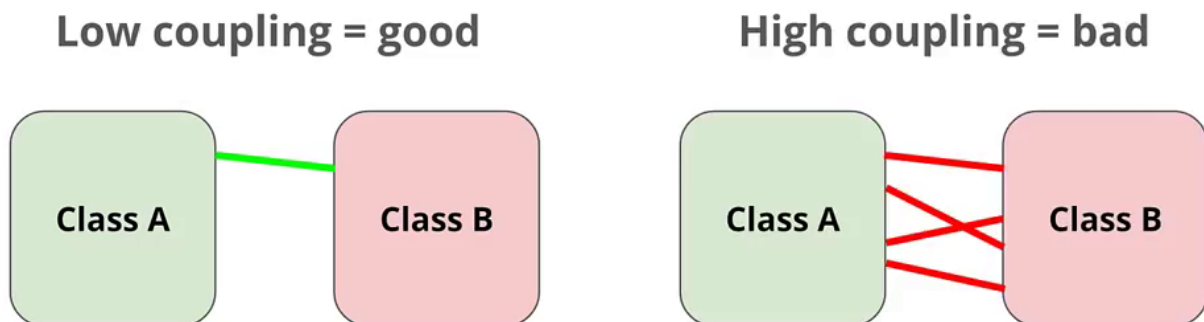
In simple terms, high-level classes should not depend directly on low-level classes. Instead, both should depend on abstractions. In Unity, this often means avoiding hard-coded references to specific implementations when an interface or event-based approach would be more flexible.

To understand why this matters, we first need to look at coupling.

What Is Coupling?

Coupling refers to how strongly one script depends on another. Some dependency is unavoidable in game development, since systems need to communicate, but the goal is to keep those dependencies as limited as possible.

Low coupling means scripts interact in a controlled and minimal way. High coupling means they are heavily intertwined, which makes the project harder to maintain as it grows.



High coupling is risky because changes in one class are more likely to break another. If Class B directly calls a function in Class A, and that function is renamed or changed, Class B may stop working until it is updated as well. In a larger Unity project, this kind of dependency can spread quickly across many scripts.

There are two especially common sources of coupling in C# scripts:

- **Calling functions directly:** When one script calls methods on another concrete class, it becomes dependent on that class's implementation.
- **Referencing or modifying variables:** Reading another script's data creates dependency, and directly changing that data from outside the class creates even tighter coupling.

Neither of these is always wrong, but both should be used carefully. The more direct connections you create between scripts, the more fragile the codebase can become.

There are two common techniques for reducing coupling in Unity projects: events and interfaces. Both help scripts communicate without becoming too dependent on specific implementations.

Events

Events allow one script to announce that something happened without needing to know which other scripts care about it. This is useful because the sender does not need direct knowledge of every listener.

For example, instead of a player script directly calling a UI script to update the score display, the player or score system could raise an event. Any interested script can subscribe to that event and respond appropriately.

This creates a looser connection. The event can still be invoked even if nothing is listening, which makes the system more flexible and easier to extend later.

Interfaces

Interfaces reduce coupling by allowing scripts to depend on capabilities rather than concrete classes. Instead of interacting with a specific implementation, a script can work with any object that satisfies a particular interface.

For example, rather than checking for a specific enemy class, a damage system might interact with any object that implements a damage-related interface. This means the underlying class can change without forcing changes in the calling code.

In practice, interfaces act as a middle layer between systems. The calling script depends on the interface definition, not on the exact class behind it. That makes your code easier to swap, test, and extend.

Summary

Taken together, the SOLID principles encourage code that is focused, modular, and easier to maintain. They also naturally help reduce coupling, which becomes increasingly important as Unity projects grow in size and complexity.

To recap, the five principles are:

1. **Single Responsibility Principle:** Each class or script should do one thing.
2. **Open-Closed Principle:** Classes should be open for extension but closed for modification.
3. **Liskov Substitution Principle:** Derived classes should be valid substitutes for their base class.
4. **Interface Segregation Principle:** Classes should not be forced to depend on methods they do not use.
5. **Dependency Inversion Principle:** High-level systems should not depend directly on low-level implementations.

Applying these principles will help you write code that is clearer, more flexible, and easier for other developers to work with. That is especially important in team environments, where well-structured systems make it much safer to add features or revise existing ones without breaking unrelated parts of the project.

In the next chapter, we will move from principles into a concrete implementation pattern and begin exploring the Singleton pattern in Unity.

OceanofPDF.com

Singleton Pattern

In the previous chapter, we looked at principles that help keep Unity code modular and maintainable. In this chapter, we will apply those ideas to one of the most common Unity patterns: the Singleton.

A singleton ensures that only one instance of a particular class exists at runtime and provides a simple way to access that instance from anywhere in your project. This can be useful for systems such as game management, audio, or saving, where a single shared controller often makes sense.

We will begin with a direct singleton implementation on a GameManager, then refactor that approach into a reusable generic base class.

Why Use a Singleton?

The singleton pattern solves a very practical problem in Unity: shared access to a unique object. Without it, scripts often rely on manually assigned references in the Inspector, which works for small setups but becomes harder to manage as a project grows.

A common example is a GameManager object in the scene that handles tasks such as score tracking, player spawning, and overall game flow. If another script needs to call a method on that manager, the traditional approach is to expose a serialized field and assign the reference manually. The player can then call the appropriate function from the Game Manager's instance reference.

```
using UnityEngine;

public class TestScript : MonoBehaviour
{
    [SerializeField] private GameManager gameManager;

    void Start()
    {
        gameManager.IncreaseScore(1);
    }
}
```

```

    }
}
using UnityEngine;

public class GameManager : MonoBehaviour
{
    public int Score { get; private set; }

    public void IncreaseScore(int amount)
    {
        Score += amount;
    }
}

```

This approach is valid, but it has some clear drawbacks:

- Every object that needs the GameManager must be linked manually.
- It is easy to forget to assign a reference.
- Runtime-spawned objects need extra setup code before they can use the manager.
- The more scripts depend on the manager, the more repetitive the Inspector wiring becomes.

A singleton removes that manual linking by giving every script a shared access point.

Implementing a Basic Singleton

We will start by adding singleton behavior directly to a GameManager. This makes the pattern easy to understand before we generalize it later in the chapter.

Adding the Instance Property

The core of a singleton is a static property that stores the one active instance. At the top of the GameManager class, add the following:

```
public static GameManager Instance { get; private set; }
```

This declaration does several things at once:

- **public** means any script can read the property.
- **static** means the property belongs to the class itself rather than to a specific object instance.
- **GameManager** is the type being stored.
- **{ get; private set; }** allows other scripts to read the value, while only GameManager can assign it.

Because the property is static, you can access it with `GameManager.Instance` from anywhere in your code.

Setting the Instance in Awake

Next, we need to assign the singleton when the object is created. Awake is the right place for this because Unity calls it early in the object lifecycle.

A first version might look like this:

```
void Awake()  
{  
    Instance = this;  
}
```

This works if there is only ever one GameManager in the scene. However, it is not safe if duplicates exist.

Unity Insight: The MonoBehaviour lifecycle

Unity calls lifecycle methods on each MonoBehaviour in a defined sequence. Awake runs first, before Start and before any frame is rendered, which makes it the right place for initialization that other scripts might rely on.

Awake runs once when the component is first loaded, even if the component starts disabled.

Start runs once on the first frame the component is active, after all Awake calls have completed. Use it for setup that depends on other objects being initialized first.

Update runs every frame while the component is active. Use it for per-frame logic such as input and movement.

Learn more: [MonoBehaviour.Awake](https://docs.unity3d.com/6000.3/Documentation/ScriptReference/MonoBehaviour.Awake.html)

(<https://docs.unity3d.com/6000.3/Documentation/ScriptReference/MonoBehaviour.Awake.html>)

Preventing Duplicate Instances

A singleton only works properly if there is exactly one active instance. So before we rely on it, we need to make sure duplicate objects are removed automatically.

If two GameManager objects exist, both will run Awake(). Without protection, the second one will overwrite the first, and the final result will depend on execution order. To avoid that, replace the simple assignment with a duplicate check:

```
void Awake()
{
    if(Instance != null && Instance != this)
    {
        Destroy(gameObject);
    }
    else
    {
        Instance = this;
    }
}
```

Here is what this logic does:

- If Instance is already assigned and it is not the current object, the current object is a duplicate and is destroyed.
- Otherwise, the current object becomes the singleton instance.

This guarantees that only one GameManager remains active at runtime.

At this stage, the complete GameManager script looks like this:

```
using UnityEngine;

public class GameManager : MonoBehaviour
{
    public static GameManager Instance { get; private set; }
    public int Score { get; private set; }

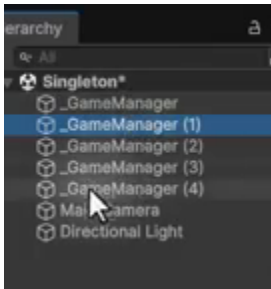
    void Awake()
    {
        if(Instance != null && Instance != this)
        {
            Destroy(gameObject);
        }
        else
        {
            Instance = this;
        }
    }

    public void IncreaseScore(int amount)
    {
        Score += amount;
    }
}
```

Testing the Singleton

Before using the pattern elsewhere, it is worth confirming that the duplicate protection behaves as expected.

To test it, duplicate the _GameManager object in the scene several times so that multiple copies appear in the Hierarchy.



When you press Play, only one GameManager should remain. The others will destroy themselves during `Awake()`, confirming that the singleton is enforcing a single active instance.

Accessing the Singleton from Other Scripts

Once the singleton is in place, other scripts no longer need a serialized field reference. Instead, they can access the manager directly through the static `Instance` property.

Update `TestScript` like this:

```
using UnityEngine;

public class TestScript : MonoBehaviour
{
    void Start()
    {
        GameManager.Instance.IncreaseScore(1);
    }
}
```

This line:

```
GameManager.Instance.IncreaseScore(1);
```

... means:

- Access the single `GameManager` instance stored in `Instance`

- Call its IncreaseScore method

The result is the same as before, but without any Inspector assignment.

When Singletons Are Useful

Singletons are most appropriate for systems that are both unique and widely needed. In Unity projects, common examples include:

- A game manager
- An audio manager
- A save system

These are systems where having a single shared controller is often reasonable.

That said, singletons should be used carefully. They create direct access points that many scripts can depend on, which increases coupling. If the GameManager API changes, every script that uses GameManager.Instance may need to be updated as well. In small projects this may be manageable, but in larger projects it can become costly.

In other words, the direct GameManager implementation works, but it does not scale well. If you later create a UIManager, AudioManager, or ScoreManager, you would need to repeat the same Instance property and Awake() logic in each class.

This repetition creates a maintenance problem:

- The same boilerplate appears in multiple scripts.
- Fixes or improvements must be copied into every singleton class.
- It becomes easier for implementations to drift out of sync.

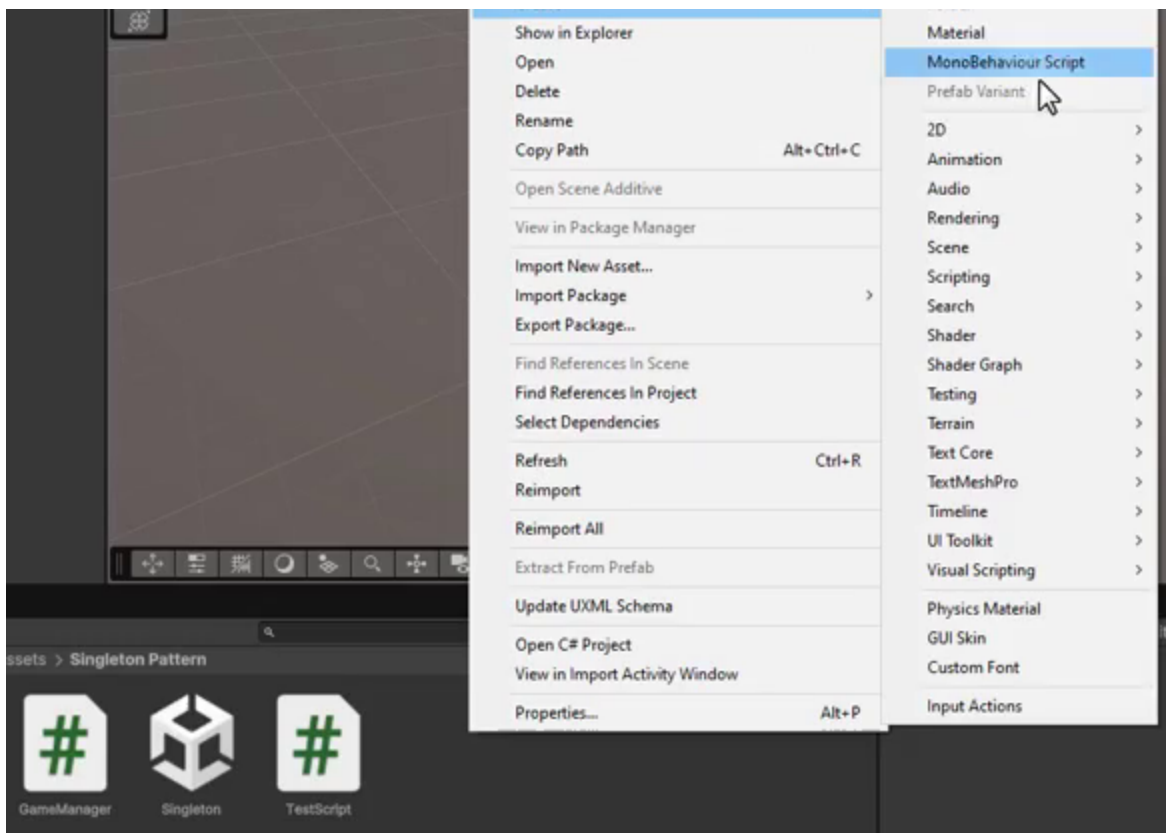
A better solution is to move the singleton logic into a reusable base class.

So while the singleton pattern is convenient, it is best reserved for systems that truly need to be global and unique.

Creating a Generic Singleton Base Class

In this section, we will build a generic Singleton<T> class. The goal is to define the singleton behavior once, then let any manager inherit from it.

In the Unity Editor, create a new MonoBehaviour script named Singleton.



Open Singleton.cs and remove the default Start() and Update() methods, since this base class will not need them.

Adding a Generic Type Parameter

To make the class reusable, we need it to work with different component types. That is what generics are for.

Change the class declaration to:

```
public class Singleton<T> : MonoBehaviour
```

Here, T is a placeholder type. When another class inherits from Singleton<T>, it will supply its own type in place of T.

Because this class is intended for Unity components, we also add a type constraint:

```
public class Singleton<T> : MonoBehaviour where T : Component
```

This ensures that T must be a Unity component type.

Next, inside the class, add the singleton property:

```
public static T Instance { get; private set; }
```

This works the same way as before, except now the property type is generic. If a class inherits from Singleton<GameManager>, then T becomes GameManager.

One detail matters a great deal here: the Instance property in Singleton.cs must be marked static. If it is not static, then code such as GameManager.Instance will not work.

Writing the Awake Method

Now let's add the singleton setup logic. We will make Awake() virtual so derived classes can override it if needed.

```
public virtual void Awake ()
{
    if(Instance != null && Instance != this)
    {
        Destroy(gameObject);
    }
    else
```

```

    {
        Instance = this as T;
    }
}

```

This is conceptually the same as the earlier GameManager version. However, there is one detail worth noting: this is not of type T. If you were to write `Instance = this`, Unity would report a compile error.

The reason is that this is a `Singleton<T>`, while `Instance` expects a value of type T. To resolve that mismatch, we cast this using `as`:

The `as` keyword performs a safe cast. If the conversion is valid, the result is assigned to `Instance`. If not, it returns null.

Using the Generic Singleton in GameManager

Now that the singleton logic lives in its own base class, GameManager can be simplified considerably.

Open GameManager.cs, remove the `Instance` property and `Awake()` method, and change the class declaration so it inherits from `Singleton<GameManager>`:

```

using UnityEngine;

public class GameManager : Singleton<GameManager>
{
    public int Score { get; private set; }

    public void IncreaseScore (int amount)
    {
        Score += amount;
    }
}

```

This is the key benefit of the generic approach: GameManager no longer contains any singleton boilerplate. It simply inherits the behavior.

Because GameManager passes itself as the generic type argument, every use of T inside `Singleton<T>` becomes `GameManager`.

Once that is in place, other scripts can continue to access the singleton in the same way as before:

```
using UnityEngine;

public class TestScript : MonoBehaviour
{
    void Start ()
    {
        GameManager.Instance.IncreaseScore(1);
    }
}
```

Reusing the Pattern for Other Managers

The generic singleton becomes especially useful once you have several manager-style systems in your project. Instead of rewriting the same logic, each manager simply inherits from the base class.

For example:

```
public class AudioManager : Singleton<AudioManager>
{
    // AudioManager code here...
}
```

The same pattern can be used for a UIManager, ScoreManager, or any other globally accessible manager. Each one gets singleton behavior automatically through inheritance.

Extending the Base Class with DontDestroyOnLoad

One of the biggest advantages of centralizing singleton logic is that changes only need to be made in one place.

For example, some singleton objects should persist across scene loads. An AudioManager or NetworkingManager often needs to remain active for the

entire game session. In that case, we can add DontDestroyOnLoad directly to the base class:

```
using UnityEngine;

public class Singleton<T> : MonoBehaviour where T : Component
{
    public static T Instance { get; private set; }

    // Destroy duplicates at startup; keep the first instance
    // alive across scene loads
    public virtual void Awake ()
    {
        if(Instance != null && Instance != this)
        {
            Destroy(gameObject);
        }
        else
        {
            // as T is required because 'this' has type
            // Singleton<T>, not T itself
            Instance = this as T;
            DontDestroyOnLoad(gameObject);
        }
    }
}
```

With this version, every class that inherits from Singleton<T> will automatically persist between scene loads. That is the real strength of the generic approach: one change in the base class updates the behavior of every singleton that uses it.

Unity Insight: DontDestroyOnLoad and scene transitions

By default, Unity destroys all GameObjects in the current scene when a new scene loads. DontDestroyOnLoad tells Unity to exempt a specific object from that cleanup, keeping it alive for the rest of the session.

This is useful for systems that need to stay active across multiple scenes. An audio manager that plays background music, a save system that tracks game state, or a networking manager that maintains a server connection are all good candidates.

One thing to be careful about: if the same singleton prefab exists in more than one scene and DontDestroyOnLoad is active, the duplicate check in Awake becomes especially important. Without it, a second instance will survive alongside the first when a scene reloads.

Learn more: [Object.DontDestroyOnLoad](https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Object.DontDestroyOnLoad.html)
(<https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Object.DontDestroyOnLoad.html>)

Summary

In this chapter, we explored the Singleton pattern in two stages.

First, we implemented a singleton directly in GameManager by:

- Adding a static Instance property
- Assigning it in Awake()
- Destroying duplicate objects to enforce uniqueness

Then we improved the design by creating a reusable generic base class:

- Singleton<T> stores the shared instance
- Derived classes inherit singleton behavior automatically
- Future changes can be made in one place instead of many

Singletons can be very useful in Unity when a system truly needs to be unique and globally accessible. At the same time, they should be used with care, since they can increase coupling if too many scripts depend on them directly.

In the next chapter, we will look at another pattern for communication between systems: the Observer pattern.

Observer Pattern

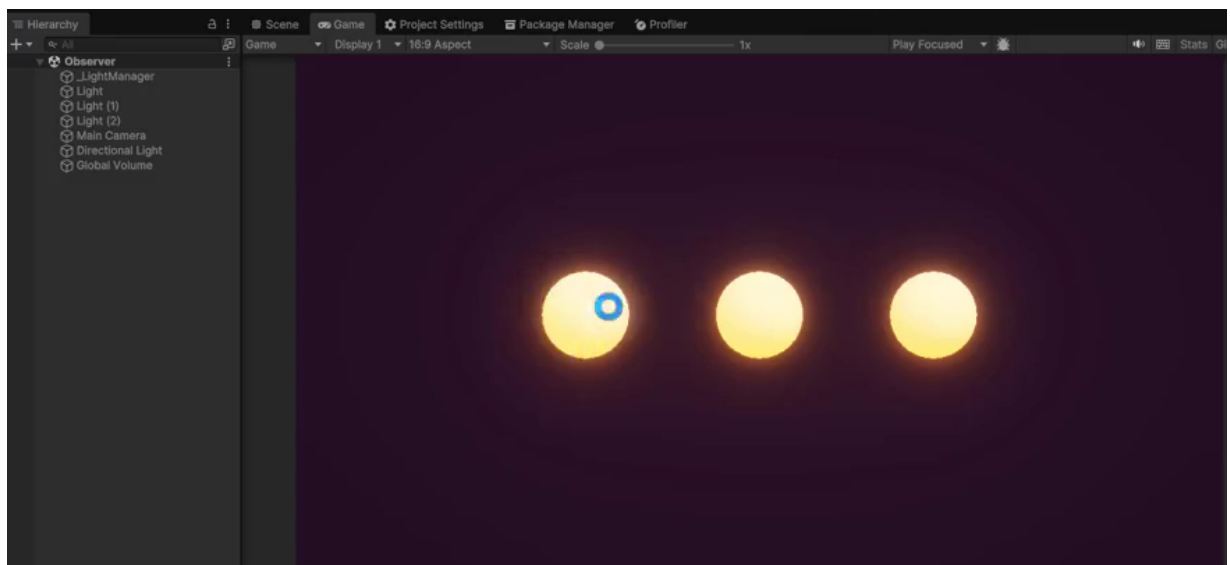
In the previous chapter, we explored the Singleton pattern as a way to provide shared access to a single object. In this chapter, we will look at another important communication pattern in Unity: the Observer pattern.

The Observer pattern allows one object to announce that something has happened while other objects listen and respond. This is especially useful when you want systems to communicate without tightly coupling them together.

In Unity, this pattern is commonly implemented with events. To demonstrate it, we will build a simple light control system where a central manager broadcasts on and off commands, and any number of lights can react automatically.

The Initial Setup

We will begin with a scene that contains three light GameObjects. Each light has a LightController script attached, and that script swaps between two materials to represent the on and off states.



The initial version of LightController is straightforward. It stores references to the renderer and the two materials, then exposes methods for turning the light on and off.

```
using UnityEngine;

public class LightController : MonoBehaviour
{
    [SerializeField] private MeshRenderer meshRend;
    [SerializeField] private Material offMaterial;
    [SerializeField] private Material onMaterial;

    public void TurnOn()
    {
        meshRend.material = onMaterial;
    }

    public void TurnOff()
    {
        meshRend.material = offMaterial;
    }
}
```

A separate LightManager object holds direct serialized references to each of the three lights. In its Update() method, it checks for keyboard input and calls TurnOn() or TurnOff() on every referenced light.

```
using UnityEngine;

public class LightManager : MonoBehaviour
{
    [SerializeField] private LightController light1;
    [SerializeField] private LightController light2;
    [SerializeField] private LightController light3;

    void Update()
    {
        if(Input.GetKeyDown(KeyCode.Alpha1))
        {
            light1.TurnOn();
            light2.TurnOn();
            light3.TurnOn();
        }
        else if(Input.GetKeyDown(KeyCode.Alpha2))
        {
            light1.TurnOff();
        }
    }
}
```

```
        light2.TurnOff();  
        light3.TurnOff();  
    }  
}
```

This works for a small example, but the design does not scale well. If you add a fourth light, you also need to add another field to `LightManager` and update both the on and off logic. As the number of lights grows, the manager becomes increasingly repetitive and harder to maintain.

What the Observer Pattern Solves

Before refactoring the code, it helps to understand the problem this pattern is designed to solve. In this case, we want one object to trigger a response in many others without needing to know exactly which objects those are.

The Observer pattern divides communication into two roles:

- **Observable objects:** objects that notify others when something changes
- **Observers:** objects that subscribe to those notifications and react when they occur

In Unity, events are a natural way to implement this pattern. An observable object exposes an event, and observer objects subscribe to it using the `+=` operator. When the event is invoked, every subscribed method is called automatically.

The key benefit is decoupling. The observable object does not need direct references to its observers. It simply raises the event, and any interested object can respond.

Refactoring LightManager with Events

We will now convert `LightManager` from a direct controller into an event broadcaster. The goal is to let the manager announce “turn on” or “turn off”

without knowing how many lights exist.

To do that, add the `UnityEngine.Events` namespace and replace the direct light references with two events of type `UnityAction`:

```
using UnityEngine;
using UnityEngine.Events;

public class LightManager : MonoBehaviour
{
    public event UnityAction OnTurnOn;
    public event UnityAction OnTurnOff;

    void Update()
    {
        if(Input.GetKeyDown(KeyCode.Alpha1))
        {
            OnTurnOn?.Invoke();
        }
        else if(Input.GetKeyDown(KeyCode.Alpha2))
        {
            OnTurnOff?.Invoke();
        }
    }
}
```

This version of `LightManager` no longer stores references to individual lights. Instead, it exposes two events:

- `OnTurnOn`, fired when the 1 key is pressed
- `OnTurnOff`, fired when the 2 key is pressed

There are two details here worth noting.

First, the event keyword matters. Without it, outside classes could invoke or overwrite the delegate directly. With event, outside code can only subscribe or unsubscribe, which is exactly what we want.

Second, the null-conditional operator in `OnTurnOn?.Invoke()` and `OnTurnOff?.Invoke()` is a safety check. If no objects have subscribed yet,

invoking the event directly would cause an error. The `?.` ensures the event is only invoked if it has at least one listener.

Unity Insight: UnityAction and the event keyword

UnityAction is a zero-argument delegate type provided by Unity's `UnityEngine.Events` namespace. It is equivalent to the built-in C# `Action` delegate, but is specifically designed to work with Unity's event system and the Inspector.

The event keyword in C# restricts how a delegate can be used from outside the class. Without it, external scripts could invoke the delegate or overwrite all its subscribers directly. With event, outside code can only add (`+=`) or remove (`-=`) listeners. This is the correct access pattern for the Observer relationship: observers subscribe, but they cannot fire or reset the event themselves.

The null-conditional `?.Invoke()` pattern is the recommended way to invoke events in C#. It checks in a single thread-safe step whether any listeners are subscribed before invoking, which prevents a `NullReferenceException` when no one is listening yet.

Learn more: [UnityAction](https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Events.UnityAction.html)

(<https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Events.UnityAction.html>)

Testing the Events

At this stage, the code is valid, but the lights still will not respond. That is expected, because nothing has subscribed to the events yet.

If you enter Play mode and press 1 or 2, there should be no errors, but there will also be no visible change in the scene.



The manager is now broadcasting events successfully. The next step is to let each light listen for those events.

Adding a Singleton to LightManager

For LightController to subscribe to the manager's events, it needs a way to access the active LightManager instance. In this example, we will use a simple singleton directly on LightManager.

This keeps the setup easy to follow and avoids assigning the manager manually in the Inspector. Add the following singleton implementation:

```

using UnityEngine;
using UnityEngine.Events;

public class LightManager : MonoBehaviour
{
    public event UnityAction OnTurnOn;
    public event UnityAction OnTurnOff;

    public static LightManager Instance { get; private set; }

    void Awake()
    {
        if(Instance != null && Instance != this)
        {
            Destroy(gameObject);
        }
        else
        {
            Instance = this;
        }
    }

    void Update()
    {
        if(Input.GetKeyDown(KeyCode.Alpha1))
        {
            OnTurnOn?.Invoke();
        }
        else if(Input.GetKeyDown(KeyCode.Alpha2))
        {
            OnTurnOff?.Invoke();
        }
    }
}

```

This is the same singleton structure introduced in the previous chapter:

- Instance stores the one active LightManager
- Awake() assigns the instance
- Duplicate managers destroy themselves automatically

With this in place, any LightController can access the manager through LightManager.Instance.

Subscribing to Events in LightController

Now we can turn each light into an observer. The idea is simple: when a light becomes active, it subscribes to the manager's events, and when it becomes inactive, it unsubscribes.

Unity's `OnEnable()` and `OnDisable()` methods are well suited to this. They let the subscription lifecycle match the component's active state, which helps prevent invalid event calls.

Update `LightController` like this:

```
using UnityEngine;

public class LightController : MonoBehaviour
{
    [SerializeField] private MeshRenderer meshRend;
    [SerializeField] private Material offMaterial;
    [SerializeField] private Material onMaterial;

    // Subscribe when the light becomes active
    void OnEnable()
    {
        LightManager.Instance.OnTurnOn += TurnOn;
        LightManager.Instance.OnTurnOff += TurnOff;
    }

    // Unsubscribe to prevent calls on a disabled or destroyed
    object
    void OnDisable()
    {
        LightManager.Instance.OnTurnOn -= TurnOn;
        LightManager.Instance.OnTurnOff -= TurnOff;
    }

    void TurnOn()
    {
        meshRend.material = onMaterial;
    }

    void TurnOff()
    {
        meshRend.material = offMaterial;
    }
}
```

```
    }  
}
```

A few important changes have happened here:

- OnEnable() subscribes TurnOn and TurnOff to the manager's events
- OnDisable() removes those subscriptions
- TurnOn() and TurnOff() are no longer public, because they are no longer called directly from outside the class

Unsubscribing in OnDisable() is good practice. If an object is disabled or destroyed but remains subscribed, the event may later try to call a method on an invalid target, which can lead to errors.

Fixing the Script Execution Order

At this point, the overall design is correct, but running the scene will produce a runtime error. The issue is not with the event logic itself. It comes from initialization order.

LightController.OnEnable() tries to access LightManager.Instance, but Unity does not guarantee that LightManager.Awake() has already run. If OnEnable() executes first, Instance is still null.

To solve this, we need LightManager to initialize slightly earlier than the default script order. Unity provides the [DefaultExecutionOrder] attribute for exactly this purpose.

Add the attribute above the LightManager class:

```
using UnityEngine;  
using UnityEngine.Events;  
  
// Run Awake before default order so Instance is ready when  
// observers subscribe  
[DefaultExecutionOrder(-1)]  
public class LightManager : MonoBehaviour  
{
```

```

public event UnityAction OnTurnOn;
public event UnityAction OnTurnOff;

public static LightManager Instance { get; private set; }

void Awake()
{
    if(Instance != null && Instance != this)
    {
        Destroy(gameObject);
    }
    else
    {
        Instance = this;
    }
}

void Update()
{
    if(Input.GetKeyDown(KeyCode.Alpha1))
    {
        OnTurnOn?.Invoke();
    }
    else if(Input.GetKeyDown(KeyCode.Alpha2))
    {
        OnTurnOff?.Invoke();
    }
}
}

```

A lower execution order value means the script runs earlier. Since the default order is 0, using -1 ensures that `LightManager.Awake()` runs before the default timing of other scripts. This means the singleton instance is ready before any `LightController` tries to subscribe.

Unity Insight: Controlling script execution order

Unity does not guarantee which `MonoBehaviour` script runs first unless you tell it otherwise. By default, all scripts share the same execution order (0), so `Awake` calls can happen in any sequence.

The `[DefaultExecutionOrder]` attribute lets you assign a priority to a specific script. Lower values run earlier. Setting a manager to -1 ensures it initializes before scripts at 0, which is often all you need.

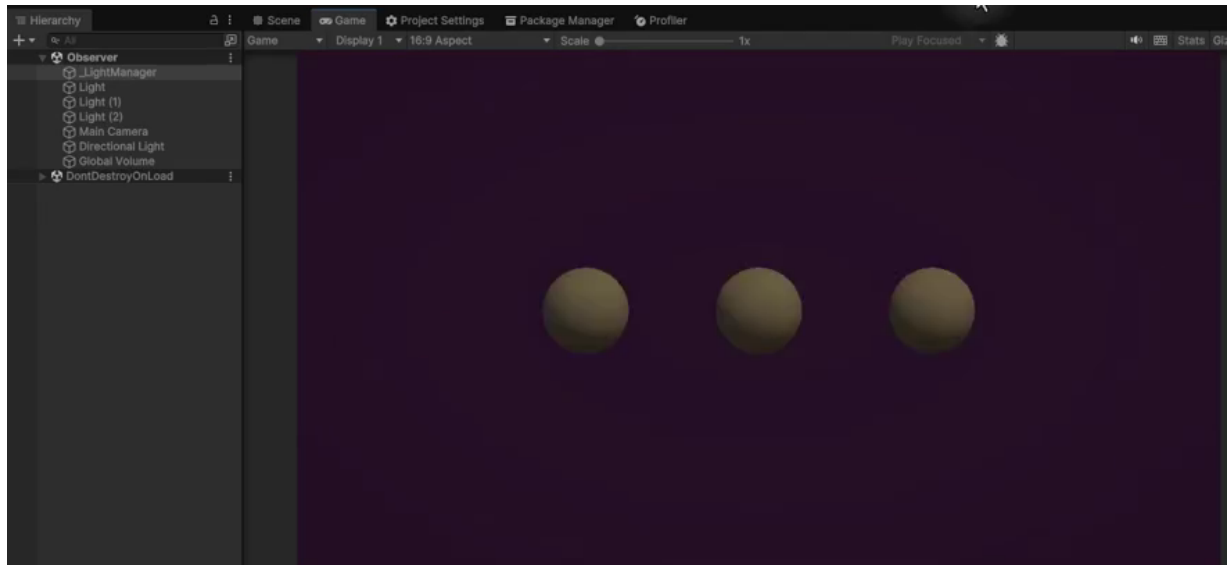
For more complex projects, Unity also provides a Script Execution Order settings panel under **Project Settings > Script Execution Order**, where you can drag scripts to define their priority visually.

Learn more: [DefaultExecutionOrder](https://docs.unity3d.com/6000.3/Documentation/ScriptReference/DefaultExecutionOrder.html)

(<https://docs.unity3d.com/6000.3/Documentation/ScriptReference/DefaultExecutionOrder.html>)

The Result: Scalable Light Control

With the observer pattern in place, the system now behaves much more cleanly. Pressing 1 turns all subscribed lights on, and pressing 2 turns them off.

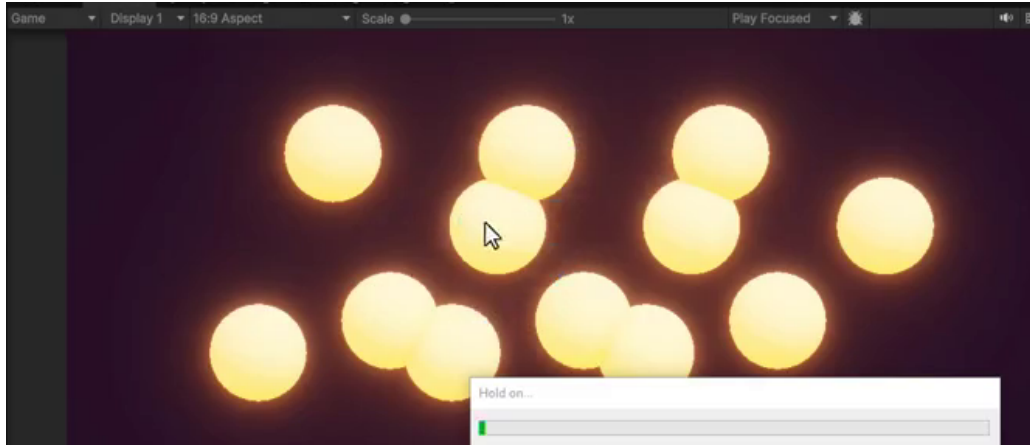


The important improvement is architectural rather than visual. LightManager no longer knows how many lights exist, and it does not need references to any of them. It simply raises events.

Each LightController is responsible for subscribing when active and unsubscribing when inactive. That means new lights can be added to the scene without changing the manager at all.

To demonstrate this, duplicate the light objects several times. The scene can contain many more lights, and they will all respond correctly without any

changes to LightManager.



This is the real strength of the Observer pattern. The manager remains simple, and the system scales naturally as observers are added or removed.

Summary

In this chapter, we used Unity events to implement the Observer pattern in a practical light control example.

We began with a tightly coupled setup where LightManager held direct references to every light. That approach worked for a few objects, but it became harder to maintain as the number of lights increased.

We then refactored the system so that:

- LightManager exposes OnTurnOn and OnTurnOff events
- LightController subscribes in OnEnable() and unsubscribes in OnDisable()
- A singleton provides shared access to the manager
- [DefaultExecutionOrder(-1)] ensures the manager initializes before observers subscribe

The result is a much more scalable design. New lights can be added freely, and the manager never needs to be updated to support them.

In the next chapter, we will continue exploring communication and behavior patterns by looking at the Command pattern in Unity.

OceanofPDF.com

Command Pattern

In the previous chapter, we used the Observer pattern to let systems communicate without tight coupling. In this chapter, we will look at another highly practical pattern for Unity game development: the Command pattern.

The Command pattern represents actions as objects. This gives you a clean way to execute, store, undo, and redo gameplay behavior. In Unity, that can be especially useful for player input, turn systems, replays, queued actions, and any feature where the order of operations matters.

To explore the pattern, we will build a simple movement system for a cube named Player. The player will move with the arrow keys, and two UI buttons, **Undo** and **Redo**, will let us step backward and forward through the command history. We will then extend the system with a second command type to show how easily the pattern scales.

Why Use the Command Pattern?

Before building the system, it helps to understand what problem this pattern solves. In many Unity projects, input handling and gameplay logic become tightly mixed together. A key press directly changes a transform, triggers an animation, or modifies game state. That works for simple prototypes, but it becomes limiting once you need features such as undo, replay, or action queues.

The Command pattern improves this by wrapping each action in its own object. Instead of saying “move now” directly, the game creates a command object that knows how to perform that move and how to reverse it later.

This structure provides several benefits:

- Actions become reusable and easier to organize

- Input handling becomes separated from the logic that performs the action
- Undo and redo become much easier to implement
- New action types can be added without rewriting the whole system

This pattern is especially useful in RTS games, turn-based games, and replay systems, but the underlying idea applies to many kinds of Unity projects.

The Example Scene

We will use a very simple scene so the pattern itself stays easy to follow. The scene contains a cube named **Player** placed in the center, along with two UI buttons at the bottom of the screen for undoing and redoing actions.



The goal is straightforward: pressing the arrow keys creates movement commands, and the buttons let us walk backward or forward through those

recorded actions.

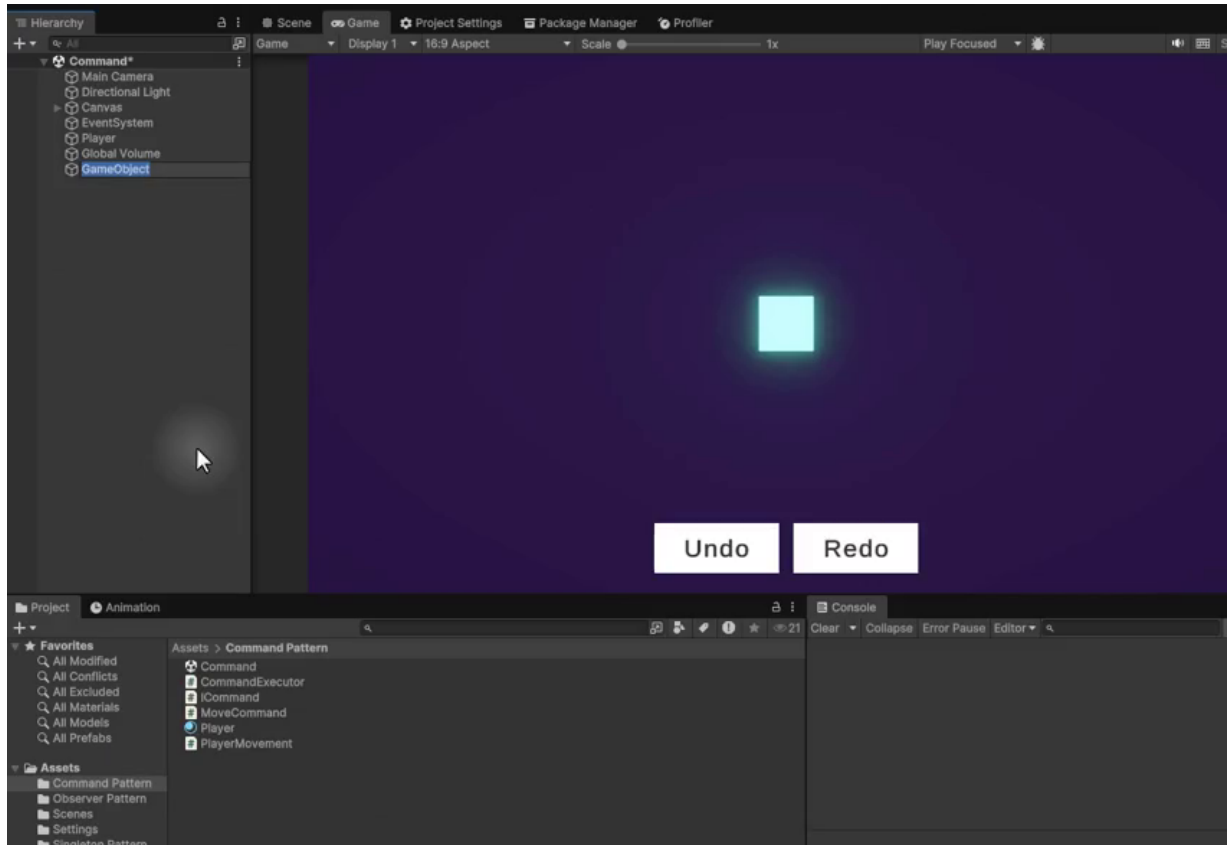
Scene Setup and Scripts

To build the system, we will create four scripts. Each one has a focused responsibility, which keeps the pattern easy to understand and extend.

We will use:

- **ICommand**: The interface that defines what every command must do
- **MoveCommand**: A concrete command that moves the player
- **PlayerMovement**: The script that performs the actual movement
- **CommandExecutor**: The script that creates, executes, and tracks commands

Attach the **PlayerMovement** script to the **Player** **GameObject**. Then create an empty **GameObject** named **CommandExecutor** and attach the **CommandExecutor** script to it.



With the scene prepared, we can begin with the abstraction that all commands will share.

Defining the ICommand Interface

The foundation of the pattern is the ICommand interface. In this case, we want every command to support two operations: performing the action and reversing it.

Open ICommand.cs, remove MonoBehaviour, and define the interface like this:

```
using UnityEngine;

public interface ICommand
{
    void Execute();
    void Undo();
}
```

This interface establishes a simple contract:

- Execute() performs the action
- Undo() reverses it

That contract is what makes the pattern flexible. A movement command, a scale command, a color-change command, or any other gameplay action can all fit into the same system as long as they implement these two methods.

Creating the MoveCommand Class

Now we need a concrete command that actually does something. MoveCommand will store the information needed to move the player, then use that information when Execute() or Undo() is called.

Because this is a plain C# class and not a component attached to a GameObject, it does not inherit from MonoBehaviour. Instead, it implements ICommand.

```
using UnityEngine;
```

```
public class MoveCommand : ICommand
{
    private PlayerMovement playerMovement;
    private Vector3 movement;

    public MoveCommand (PlayerMovement playerMovement, Vector3
movement)
    {
        this.playerMovement = playerMovement;
        this.movement = movement;
    }

    public void Execute ()
    {
        playerMovement.Move(movement);
    }

    public void Undo ()
    {

```

```

        // Reverse the move by applying the opposite direction
        playerMovement.Move(-movement);
    }
}

```

This class stores two pieces of data:

- A reference to PlayerMovement, so it knows which object to affect
- A Vector3 movement value, so it knows how far and in what direction to move

The constructor receives those values when the command is created. Then:

- Execute() applies the stored movement
- Undo() applies the opposite movement by negating the vector

This is one of the strengths of the pattern: the command object contains everything it needs to carry out and reverse its own behavior.

Creating the PlayerMovement Script

The command should describe the action, but another script should perform the actual movement. This keeps responsibilities separate and makes the command easier to reuse.

Attach the following script to the Player object:

```

using UnityEngine;

public class PlayerMovement : MonoBehaviour
{
    public void Move (Vector3 movement)
    {
        transform.Translate(movement);
    }
}

```

Transform.Translate adds the given vector to the current position. For this example, that makes movement simple and predictable.

Unity Insight: Transform.Translate and coordinate space

Transform.Translate moves an object by adding a vector to its current position. By default it moves in **local space**, meaning the direction is relative to the object's own orientation. For example, Vector3.forward always means "forward from the object's perspective."

When you pass a second argument of Space.World, the movement is interpreted in world space instead. For this example, the player always moves along world axes (up, down, left, right), so local and world space produce the same result because the player is not rotated.

Learn more: [Transform.Translate](https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Transform.Translate.html)

(<https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Transform.Translate.html>)

At this point, MoveCommand knows how to request movement, and PlayerMovement knows how to apply it. What we still need is a central script to create commands, execute them, and track history.

Building the CommandExecutor

The CommandExecutor is the part that ties the whole pattern together. It will listen for input, create command objects, execute them, and store them so they can be undone or redone later.

We will begin by declaring the references and data structures the system needs.

```
using UnityEngine;
using System.Collections.Generic;

public class CommandExecutor : MonoBehaviour
{
    [SerializeField] private PlayerMovement playerMovement;
    private Stack<ICommand> undoStack = new Stack<ICommand>();
    private Stack<ICommand> redoStack = new Stack<ICommand>();
}
```

There are three important fields here:

- `playerMovement`: a serialized reference assigned in the Inspector
- `undoStack`: stores commands that have been executed and can be undone
- `redoStack`: stores commands that have been undone and can be redone

The `Stack<T>` type is useful because undo and redo naturally follow a last-in, first-out structure. The most recent action is the first one you usually want to reverse.

You can think of a stack like a pile of objects. New items are placed on top, and when you remove one, you always take the top item first. That behavior matches undo systems very well.

Handling Player Input

Now we can start turning player input into command objects. In this section, the goal is to detect arrow key presses and create a `MoveCommand` for each one.

Add the following `Update()` method to `CommandExecutor`:

```
void Update()
{
    if(Input.GetKeyDown(KeyCode.UpArrow))
        ExecuteCommand(new MoveCommand(playerMovement,
Vector3.up));

    if(Input.GetKeyDown(KeyCode.DownArrow))
        ExecuteCommand(new MoveCommand(playerMovement,
Vector3.down));

    if(Input.GetKeyDown(KeyCode.LeftArrow))
        ExecuteCommand(new MoveCommand(playerMovement,
Vector3.left));

    if(Input.GetKeyDown(KeyCode.RightArrow))
        ExecuteCommand(new MoveCommand(playerMovement,
```

```
Vector3.right));  
}
```

Each key press creates a new `MoveCommand` object and passes it to `ExecuteCommand`. Notice that `CommandExecutor` does not move the player directly. Instead, it creates a command that knows how to do that work.

That separation is the core idea of the pattern.

Executing Commands and Recording History

Once a command is created, we need to do three things:

1. Execute it
2. Store it in the undo history
3. Clear any invalid redo history

Add the following method to `CommandExecutor`:

```
void ExecuteCommand(ICommand command)  
{  
    command.Execute();  
    undoStack.Push(command);  
    // Discard the redo chain: a new action invalidates  
previous undone commands  
    redoStack.Clear();  
}
```

This method is small, but it defines the behavior of the whole system.

First, `command.Execute()` performs the action. Then the command is pushed onto `undoStack`, which means it is now available to undo later. Finally, `redoStack.Clear()` removes any previously undone commands.

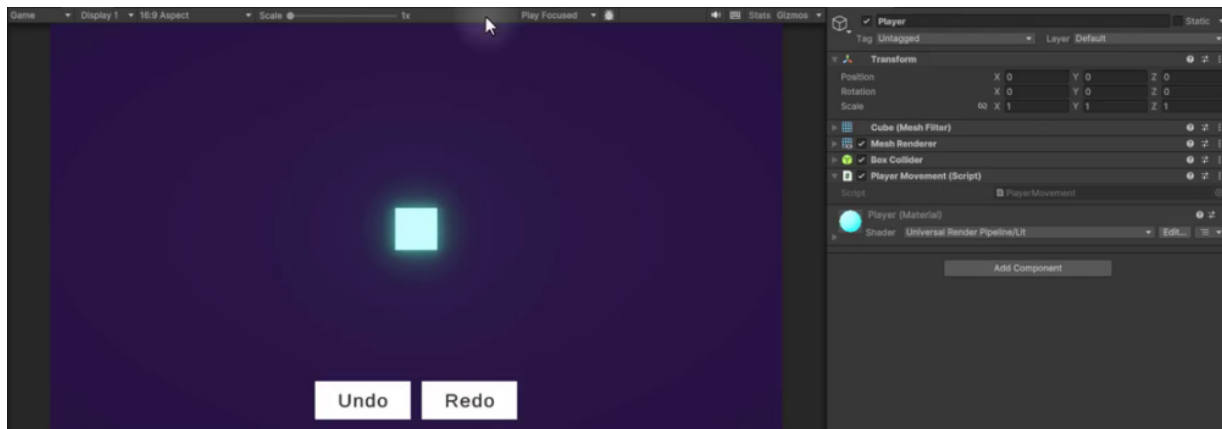
That last step matters. If the player undoes several actions and then performs a brand new action, the old redo path is no longer valid. The

history has branched in a new direction, so the previous redo chain must be discarded.

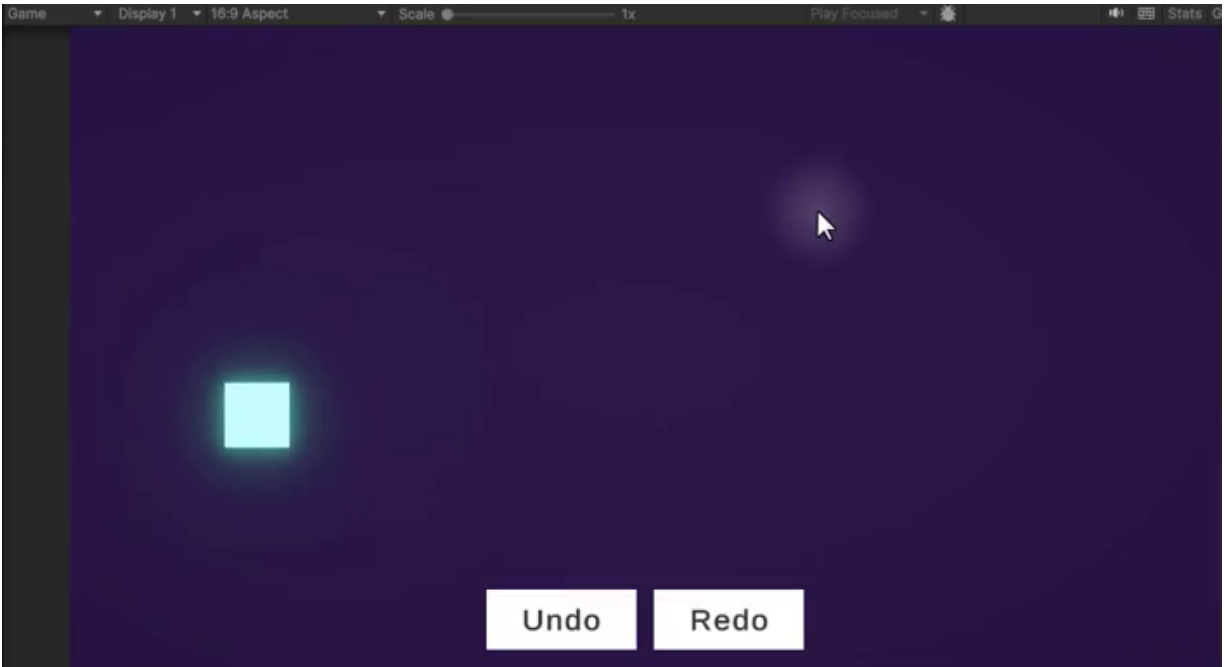
Testing Basic Movement

Before adding undo and redo, it is worth confirming that command execution is working correctly.

In the Unity Editor, make sure the CommandExecutor object has the CommandExecutor script attached, then drag the Player object into its playerMovement field in the Inspector. Also confirm that the PlayerMovement script is attached to the player itself.



Enter Play mode and press the arrow keys. The player should move around the scene, and each movement command will be stored in the undo stack.



With command execution working, we can now add the ability to reverse those actions.

Implementing Undo

Undo should reverse the most recent command. To do that safely, we first need to check whether there is anything in the undo stack.

Add this method to `CommandExecutor`:

```
public void UndoCommand()
{
    if(undoStack.Count == 0)
        return;

    ICommand currentCommand = undoStack.Pop();
    // Move to the redo stack so this command can be replayed
    if needed
        redoStack.Push(currentCommand);
    currentCommand.Undo();
}
```

This method works in three steps:

- If there are no commands to undo, return immediately
- Pop the most recent command from undoStack
- Push that command onto redoStack, then call Undo()

Pushing the command onto redoStack is what makes redo possible later. Once an action has been undone, it becomes a candidate to replay.

Implementing Redo

Redo is the reverse of undo. Instead of stepping backward through history, it replays the most recently undone command.

Add the following method:

```
public void RedoCommand()
{
    if(redoStack.Count == 0)
        return;

    ICommand currentCommand = redoStack.Pop();
    // Return to the undo stack so the replayed command can be
    undone again
    undoStack.Push(currentCommand);
    currentCommand.Execute();
}
```

This follows the same pattern:

- If there is nothing to redo, return
- Pop the command from redoStack
- Push it back onto undoStack
- Call Execute() to replay the action

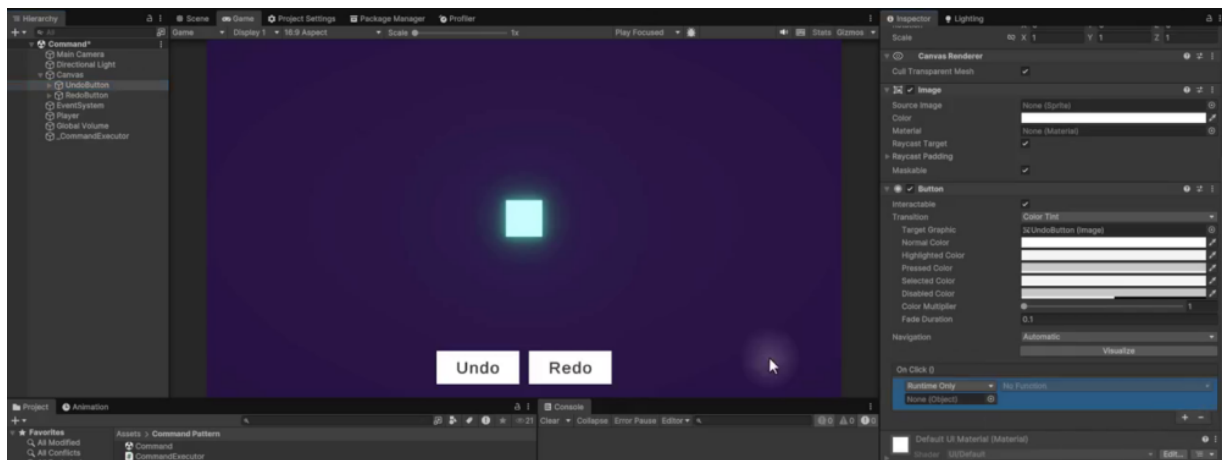
Together, UndoCommand() and RedoCommand() let the player move backward and forward through the command history.

Connecting the UI Buttons

The command system is now complete in code, so the final setup step is to connect the UI buttons to the public methods we just created.

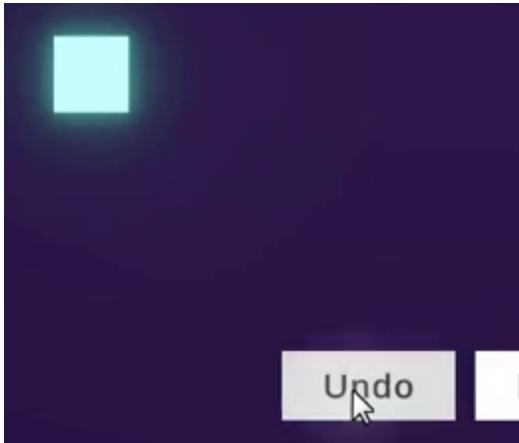
Select the **Undo** button in the Hierarchy and go to its **On Click ()** event list in the Inspector. Add a new listener, drag the CommandExecutor GameObject into the object field, and choose `CommandExecutor.UndoCommand`.

Repeat the same process for the **Redo** button, this time selecting `CommandExecutor.RedoCommand`.



Enter Play mode again and test the full system:

- Move the player with the arrow keys
- Click **Undo** to reverse the most recent actions one by one
- Click **Redo** to replay undone actions



One important behavior to remember is this: if you undo several commands and then execute a new command, the redo stack is cleared. That is expected, because the action history has changed.

Practical Uses of the Command Pattern

Now that the system is working, it is worth stepping back and considering where this pattern is useful in real projects. The example here is intentionally simple, but the same structure can support much more advanced systems.

Common use cases include:

- **Turn-based games:** Collect commands during a turn, then execute them in sequence
- **Multiplayer games:** Rewind and replay command history when correcting state
- **RTS and strategy games:** Queue, cancel, or reorder unit actions
- **Replay systems:** Record commands and play them back later

The pattern is valuable because it treats actions as data. Once an action is represented as an object, it becomes much easier to store, reorder, repeat, or reverse.

Extending the System with a New Command

A major advantage of the Command pattern is extensibility. Once the infrastructure is in place, adding a new action type usually requires very little change to the existing system.

To demonstrate that, we will add a ScaleCommand that increases the player's size. This follows the same structure as MoveCommand, which makes it a good example of how reusable the pattern becomes.

Creating the ScaleCommand Script

Start by creating a new C# script named ScaleCommand. Like MoveCommand, this will be a plain C# class that implements ICommand.

Begin with the following structure:

```
public class ScaleCommand : ICommand
{
    public void Execute()
    {
    }

    public void Undo()
    {
    }
}
```

At this stage, the class satisfies the interface but does not yet contain the data it needs. We will add that next.

Adding Scale Support to PlayerMovement

Before ScaleCommand can do anything, PlayerMovement needs a method that applies a scale change. In this case, we want to keep the same style as

the movement logic: the command passes in a vector, and PlayerMovement applies it.

Add this method to PlayerMovement.cs:

```
public void Scale(Vector3 scaleAmount)
{
    transform.localScale += scaleAmount;
}
```

This method increases the object's local scale by the supplied amount. It mirrors the existing Move() method closely, which helps keep the design consistent.

Completing the ScaleCommand Class

Now we can return to ScaleCommand and give it the data it needs. Just like MoveCommand, it should store a reference to PlayerMovement and the amount of change to apply.

Update the script like this:

```
public class ScaleCommand : ICommand
{
    private PlayerMovement playerMovement;
    private Vector3 scaleAmount;

    public ScaleCommand(PlayerMovement playerMovement, Vector3
scaleAmount)
    {
        this.playerMovement = playerMovement;
        this.scaleAmount = scaleAmount;
    }

    public void Execute()
    {
    }

    public void Undo()
    {
    }
}
```

Then implement the two interface methods:

```
public void Execute()  
{  
    playerMovement.Scale(scaleAmount);  
}  
  
public void Undo()  
{  
    // Reverse the scale by applying the opposite amount  
    playerMovement.Scale(-scaleAmount);  
}
```

This works for the same reason `MoveCommand` works: the action can be reversed by applying the opposite vector.

Triggering ScaleCommand from Input

With the new command class complete, we only need one small addition to `CommandExecutor`. We want the **S** key to create and execute a `ScaleCommand`.

Add this condition inside `Update()`:

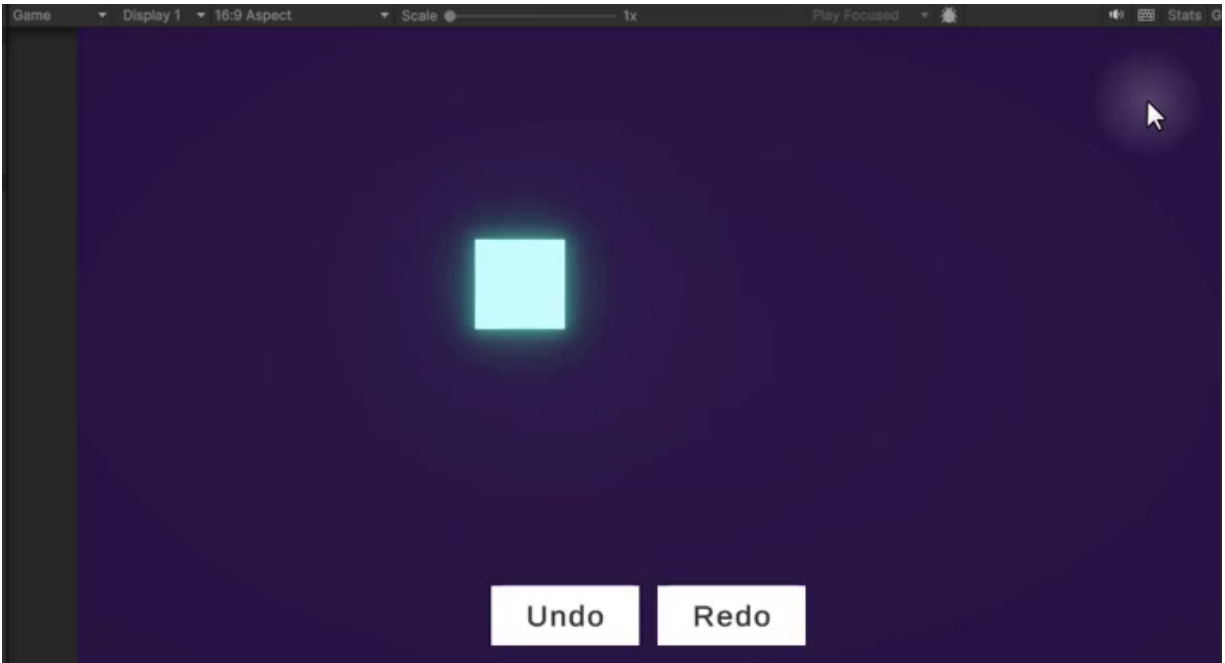
```
if(Input.GetKeyDown(KeyCode.S))  
    ExecuteCommand(new ScaleCommand(playerMovement, new  
Vector3(0.2f, 0.2f, 0.2f)));
```

No other changes are required. That is the key benefit of the pattern: `ExecuteCommand`, `UndoCommand`, and `RedoCommand` already work with any object that implements `ICommand`.

So once `ScaleCommand` follows the same interface, it automatically becomes part of the existing undo and redo system.

Testing the Extended System

Return to Unity and enter Play mode. The arrow keys should still move the player as before. Pressing **S** should now increase the player's scale by 0.2 on all three axes.



Try mixing movement and scaling actions in any order. Then use **Undo** and **Redo** to step backward and forward through the full sequence. Because both commands implement `ICommand` , the history system treats them the same way.

This is a strong demonstration of the pattern's flexibility. We added a completely new action type with:

- One new command class
- One new method in `PlayerMovement`
- One new input check in `CommandExecutor`

The existing infrastructure did not need to be rewritten.

Summary

In this chapter, we built a complete Command pattern workflow in Unity.

We began by defining an `ICommand` interface with `Execute()` and `Undo()` methods. From there, we created a `MoveCommand` that stores movement

data and uses `PlayerMovement` to apply or reverse it.

We then introduced `CommandExecutor`, which:

- Creates commands from player input
- Executes them
- Stores them in an undo stack
- Moves undone commands into a redo stack

After that, we extended the system with a `ScaleCommand`, showing how easily new actions can plug into the same infrastructure.

The key idea of the Command pattern is simple: represent actions as objects. Once you do that, features such as undo, redo, replay, and action queues become much easier to build in a clean and maintainable way.

In the next chapter, we will move on to the final pattern in this book: object pooling.

OceanofPDF.com

Object Pooling

In the previous chapter, we explored the Command pattern and saw how actions can be represented as reusable objects. In this final chapter, we will look at a different kind of optimization pattern: object pooling.

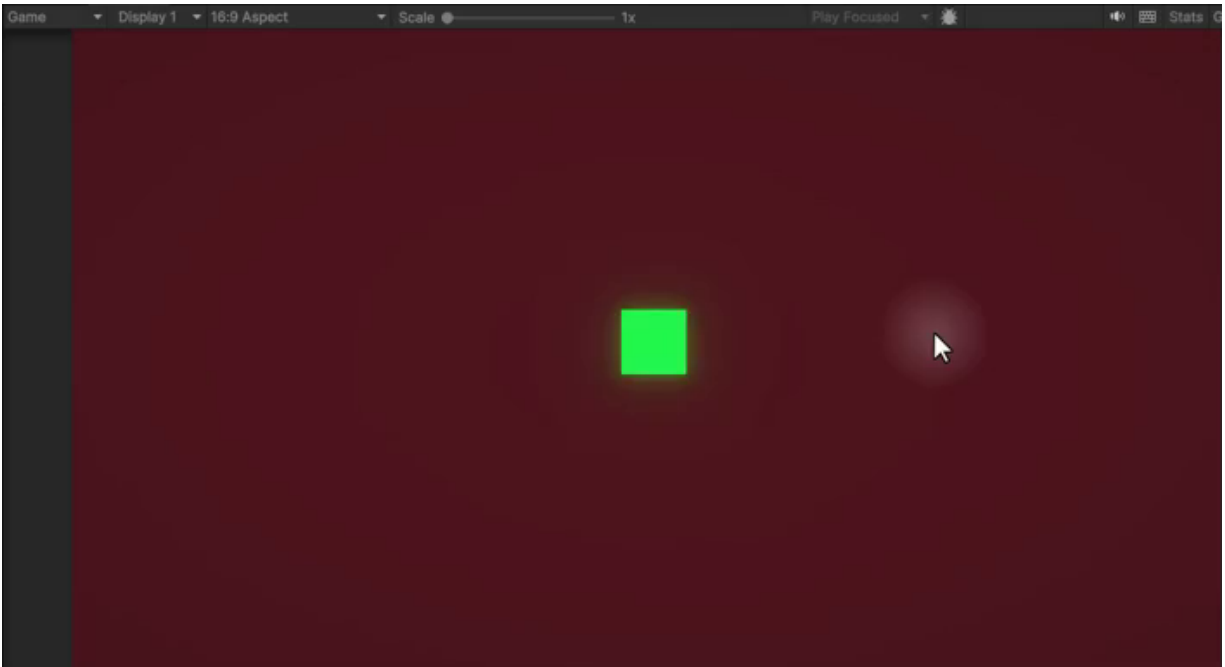
Object pooling is especially useful in Unity games that repeatedly create and remove large numbers of similar objects, such as bullets, enemies, or particle effects. Instead of constantly calling `Instantiate` and `Destroy` during gameplay, we create a reusable pool of objects ahead of time and activate or deactivate them as needed.

Why Object Pooling Matters

Before building the pool itself, it helps to understand the problem it solves. In Unity, creating and destroying objects at runtime is convenient, but doing it too often can become expensive.

Every time `Instantiate` is called, Unity creates a new object in memory and updates the systems that need to know about it. A single call is usually fine, but if your game is spawning dozens or hundreds of objects every second, those costs add up quickly. The same is true for `Destroy`, which also creates overhead and contributes to memory churn.

Our example project demonstrates this clearly. It is a small top-down scene where the player moves with the arrow keys and fires projectiles in random directions by holding the spacebar.



The Shooter script handles movement in Update() and calls a Shoot() method whenever the player fires. In the initial version, that method creates a brand-new projectile every time:

```
using UnityEngine;
```

```
public class Shooter : MonoBehaviour
{
    [SerializeField] private GameObject projectilePrefab;
    [SerializeField] private float shootRate;
    private float lastShootTime;

    void Update ()
    {
        // Movement
        Vector2 moveInput = new
        Vector2(Input.GetAxisRaw("Horizontal"),
        Input.GetAxisRaw("Vertical")).normalized;
        transform.Translate(moveInput * 2.0f * Time.deltaTime);

        // Shooting
        if(Input.GetKey(KeyCode.Space))
        {
            if(Time.time - lastShootTime > shootRate)
            {
                Shoot();
            }
        }
    }
}
```

```

        }
    }
}

void Shoot ()
{
    lastShootTime = Time.time;

    GameObject proj = Instantiate(projectilePrefab,
transform.position, Quaternion.identity);

    Vector3 shootDirection = new
Vector3(Random.Range(-1.0f, 1.0f), Random.Range(-1.0f, 1.0f),
0).normalized;
    proj.GetComponent<Projectile>
().Initialize(shootDirection);
}
}

```

Each projectile then applies velocity through its Rigidbody and destroys itself after a short lifetime.

```

using UnityEngine;

public class Projectile : MonoBehaviour
{
    [SerializeField] private float speed;
    [SerializeField] private float lifetime;

    private Rigidbody rig;

    void Awake ()
    {
        rig = GetComponent<Rigidbody>();
    }

    public void Initialize (Vector3 direction)
    {
        rig.linearVelocity = direction * speed;

        Destroy(gameObject, lifetime);
    }
}

```

If the fire rate is around 20 projectiles per second, then Instantiate and Destroy are each being called 20 times every second. If multiple enemies

are also firing, that number can rise dramatically.

Object pooling improves this by shifting most of that work to startup. We pre-create a set of objects, keep them inactive, and reuse them instead of constantly allocating and destroying new ones.

Unity Insight: The cost of Instantiate and Destroy

Object.Instantiate does more than allocate memory. Unity must register the new object with the physics engine, the rendering pipeline, and any other systems that track active GameObjects. Destroy triggers a similar process in reverse, plus garbage collection overhead from freeing the old object's memory.

These costs are affordable for occasional use, but calling either function dozens of times per second creates measurable frame spikes, especially on mobile hardware or in scenes with many active objects.

Object pooling sidesteps most of this by keeping objects in memory and toggling their active state instead. SetActive(false) effectively hides an object and removes it from per-frame processing, while SetActive(true) brings it back, all without the allocation overhead of a fresh Instantiate call.

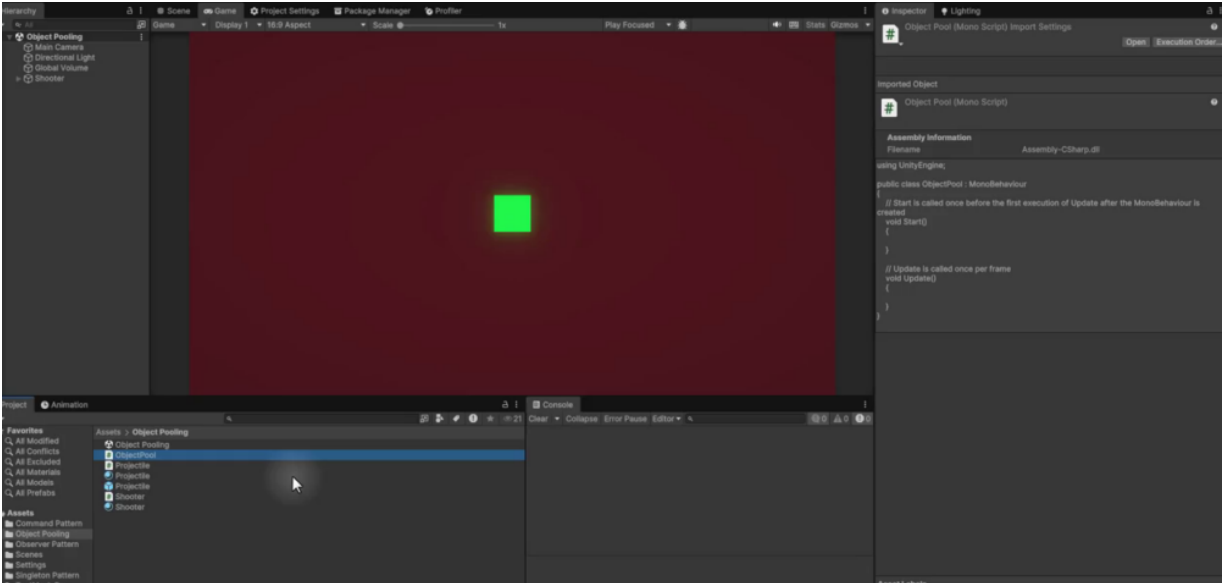
Learn more: [Object.Instantiate](https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Object.Instantiate.html)

(<https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Object.Instantiate.html>)

Building a Reusable ObjectPool Script

We will begin by creating the pooling system itself. In this case, we want the script to be generic so it can manage any GameObject prefab, not just projectiles.

Create a new C# script named ObjectPool.



To support a list of pooled objects, add the `System.Collections.Generic` namespace. Then remove the default `Start()` and `Update()` methods and define the core fields:

```
using System.Collections.Generic;
using UnityEngine;

public class ObjectPool : MonoBehaviour
{
    [SerializeField] private GameObject prefab;
    [SerializeField] private float minimumAmount;

    private List<GameObject> pool = new List<GameObject>();
}
```

These fields each have a specific role:

- `prefab` defines which object this pool will create and manage
- `minimumAmount` determines how many instances should be pre-spawned at startup
- `pool` stores references to every object created by the pool

Because the script works with plain `GameObject` references, the same component can be reused for bullets, enemies, effects, or any other pooled

prefab.

Creating New Pooled Objects

Next, we need a helper method that creates one object, adds it to the pool, and deactivates it immediately. This keeps the setup logic in one place.

Add the following method:

```
GameObject CreateNewObject ()
{
    GameObject obj = Instantiate(prefab);
    pool.Add(obj);
    obj.SetActive(false);

    return obj;
}
```

This method does three things:

- Instantiates the configured prefab
- Adds the new object to the internal list
- Deactivates it so it is ready for later use

Notice that no position or rotation is supplied here. That is intentional. The pool's job is only to manage reusable objects. The script requesting an object will decide where it should appear and how it should be initialized.

Pre-Spawning the Pool in Start

Now we can populate the pool before gameplay begins. This reduces the amount of runtime instantiation needed once the game is active.

Add a Start() method that creates the initial batch of objects:

```
void Start ()
{
    for(int i = 0; i < minimumAmount; i++)
    {
```

```

        CreateNewObject();
    }
}

```

If `minimumAmount` is set to 10 in the Inspector, this loop creates 10 inactive objects as soon as the scene starts. They remain hidden and unused until another script requests them.

Retrieving Objects from the Pool

The final core feature of the pool is a public method that returns an available object. In this case, “available” means inactive.

Add the following method:

```

public GameObject GetObject ()
{
    GameObject obj = pool.Find(x => x.activeSelf == false);

    // Grow the pool automatically if all objects are currently
    active
    if(obj == null)
        obj = CreateNewObject();

    obj.SetActive(true);

    return obj;
}

```

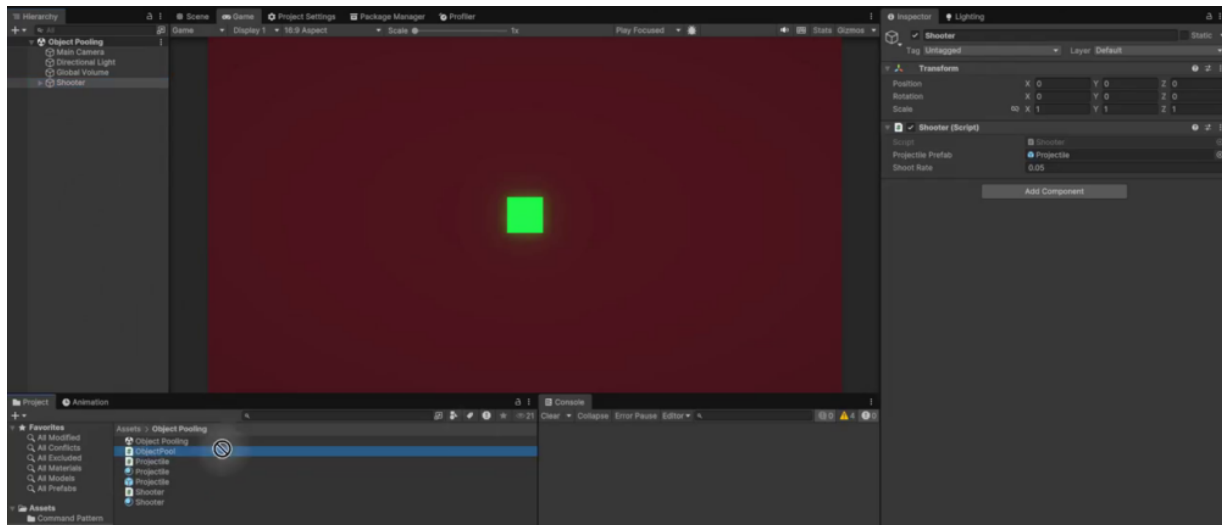
This method searches the list for the first object whose `activeSelf` value is false. If one is found, it is reactivated and returned. If all pooled objects are currently in use, the pool expands automatically by creating a new one.

That behavior is important. The pool starts with a minimum size, but it is not limited to that size. Over time, it can grow to match the actual demands of the game.

Adding the Pool to the Scene

With the script complete, we can attach it to the scene and configure it for projectiles. In this example, the `ObjectPool` component is added to the

Shooter object, the projectile prefab is assigned to the Prefab field, and Minimum Amount is set to 10.



When you press Play, the pool now creates 10 projectile objects at startup and keeps them inactive until needed. The next step is to connect the shooter to this pool so firing no longer uses Instantiate.

Refactoring Shooter to Use the Pool

Now that the pool exists, we can replace direct instantiation with pooled retrieval. This is where the gameplay code begins to benefit from the new system.

Open Shooter.cs and add a serialized field for the pool near the top of the class:

```
[SerializeField] private ObjectPool projectilePool;
```

The old projectilePrefab field is no longer needed for spawning; you can leave it or comment it out:

```
[SerializeField] private GameObject projectilePrefab; // Not  
being used anymore
```

Next, update the Shoot() method so it requests a projectile from the pool instead of creating a new one:

```
void Shoot ()
{
    lastShootTime = Time.time;

    //GameObject proj = Instantiate(projectilePrefab,
    transform.position, Quaternion.identity);
    GameObject proj = projectilePool.GetObject();

    // Reposition before use: pooled objects retain the
    transform from their last active state
    proj.transform.position = transform.position;

    Vector3 shootDirection = new Vector3(Random.Range(-1.0f,
    1.0f), Random.Range(-1.0f, 1.0f), 0).normalized;
    proj.GetComponent<Projectile>().Initialize(shootDirection);
}
```

The logic is almost the same as before, but there are two important differences:

- projectilePool.GetObject() replaces Instantiate
- The projectile's position is assigned manually after retrieval

That second step matters because Instantiate previously handled the spawn position directly. Since pooled objects already exist, we now reposition them before reusing them.

Returning Projectiles to the Pool

Pooling only works if objects are returned to the pool instead of being destroyed. So after updating the shooter, we also need to update the projectile behavior.

Open Projectile.cs. In the original version, the projectile destroys itself after a set lifetime. With pooling, we want it to deactivate itself instead.

Start by adding a private Despawn() method:

```
void Despawn ()
{
    gameObject.SetActive(false);
}
```

Then update Initialize() so it schedules Despawn() instead of calling Destroy():

```
public void Initialize (Vector3 direction)
{
    rig.linearVelocity = direction * speed;

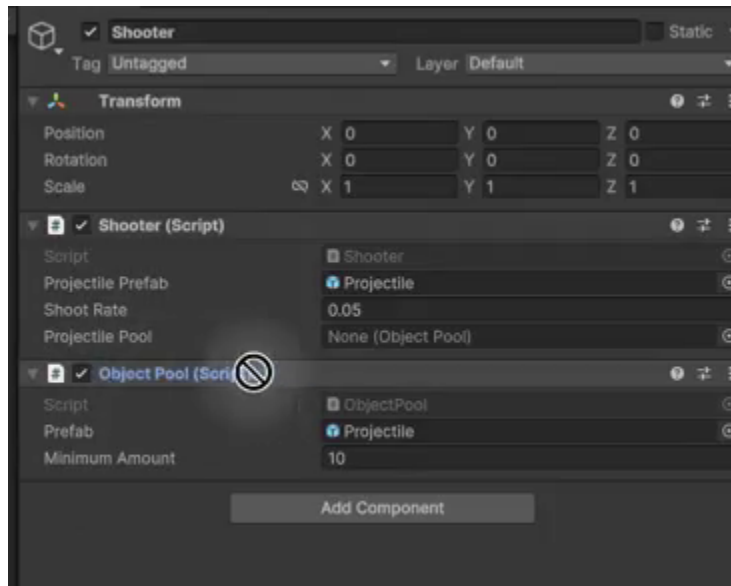
    //Destroy(gameObject, lifetime);
    // nameof avoids a fragile string literal if Despawn is
    ever renamed
    Invoke(nameof(Despawn), lifetime);
}
```

This change is small, but it is central to the pattern. The projectile is no longer removed from memory. It simply becomes inactive and waits to be reused.

Using nameof(Despawn) is also a good practice here. It avoids the fragility of hardcoded method-name strings and gives you compile-time feedback if the method is renamed later.

Wiring the Pool Reference in the Inspector

Before testing, the shooter needs a reference to the pool component. Select the Shooter object in the Hierarchy, then assign the object containing ObjectPool to the Projectile Pool field in the Inspector.



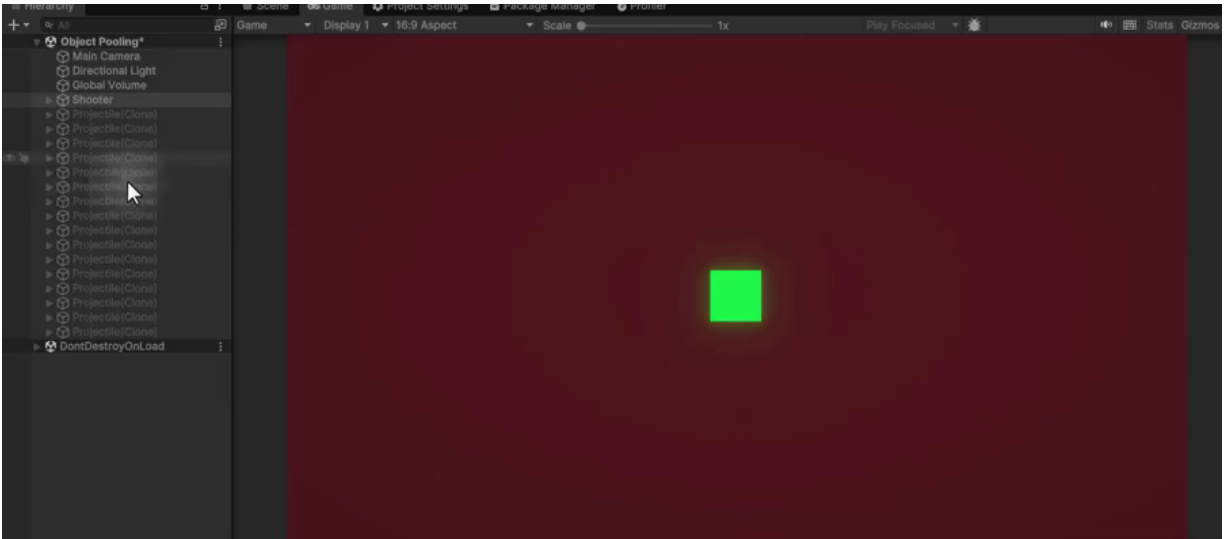
Once that reference is assigned, the system is fully connected:

- The shooter requests projectiles from the pool
- The pool activates an available object or creates one if necessary
- The projectile deactivates itself when its lifetime ends

Testing the Pooling System

At this point, the object pool is working end to end. Enter Play mode and begin firing.

Projectiles should now come from the pool rather than being instantiated every time. When a projectile expires, it deactivates itself and becomes available for reuse. If all pooled projectiles are currently active, the pool creates additional ones automatically.



Over time, the pool stabilizes at whatever size the current gameplay demands. If 10 projectiles are enough, no more are created. If the fire rate temporarily requires more, the pool expands and then continues reusing that larger set.

Performance Benefits of Object Pooling

Now that the implementation is complete, it is worth summarizing why this pattern is so useful.

Object pooling improves performance in two main ways:

- It reduces the number of Instantiate calls during gameplay
- It avoids constant memory allocation and deallocation from repeated creation and destruction

In a small prototype, the difference may be subtle. But in scenes with many bullets, enemies, or visual effects, pooling can significantly reduce frame spikes and improve runtime stability.

This pattern is also conceptually related to the Factory pattern, since both deal with object creation. The difference is that object pooling focuses

specifically on reusing existing instances rather than always creating new ones.

Summary

In this chapter, we built a complete object pooling workflow in Unity.

We began by looking at the performance cost of repeatedly using `Instantiate` and `Destroy` for fast-spawning gameplay objects such as projectiles. We then created a reusable `ObjectPool` component that:

- Stores a prefab reference
- Pre-spawns a minimum number of inactive objects
- Returns inactive objects through `GetObject()`
- Expands automatically if all pooled objects are in use

After that, we integrated the pool into the shooter system by:

- Replacing projectile instantiation with `projectilePool.GetObject()`
- Repositioning the retrieved projectile before use
- Updating `Projectile` so it deactivates itself instead of destroying itself

The result is a cleaner and more efficient spawning system that scales much better under heavy use.

With that, we have reached the final pattern in this book. Across these chapters, you have explored SOLID principles, the Singleton pattern, the Observer pattern, the Command pattern, and object pooling, all in a Unity-focused context. Together, these ideas provide a strong foundation for writing code that is more modular, more maintainable, and better suited to growing game projects.

Closing Words

Congratulations. Finishing a hands-on game development book takes real effort, and you made it all the way through. You did more than read about Unity concepts, you built with them, tested them, fixed things when they broke, and kept going. That kind of practice matters.

By this point, you have already done something many beginners never quite reach: you moved from following isolated steps to understanding how game systems fit together. That is a big milestone, and you should feel proud of it.

What You Accomplished

Before looking ahead, it is worth pausing to recognize what you now know how to do. The value of these projects is not just in the finished scenes, but in the skills you can carry into your own games.

You learned how to turn player intent into gameplay. That includes handling input, reacting to actions, and connecting controls to movement and game logic in a way that feels responsive. You also worked with physics and collisions, which means you now have experience making objects interact in a believable, rule-driven world.

You built systems that are more than one-off scripts. You practiced organizing code so that responsibilities stay clear, dependencies stay manageable, and features are easier to extend. That is where principles like SOLID become practical: they help your projects stay understandable as they grow.

You also explored communication between systems. Through events, observers, commands, and shared services, you saw different ways for game objects to react, coordinate, and stay decoupled. This is a major step forward, because scalable game development depends on systems that can talk to each other without becoming tangled.

Along the way, you touched the kinds of features that make games feel alive:

- UI that reflects game state
- Signals and event-driven behavior
- Reusable gameplay actions
- Efficient spawning and recycling of objects
- Procedural or dynamic content patterns
- Cleaner architecture for future expansion

These are not small topics. Together, they form the foundation of real game development work. Even if you still feel like a beginner, you are now a beginner with tools, vocabulary, and experience.

Where to Go From Here

This final section is about momentum. The best next step is not to start over from zero, but to build on what you already made and turn these tutorial projects into something more personal.

A great place to begin is by expanding the projects you completed. Add a new enemy type, a scoring system, power-ups, sound effects, save data, or a main menu. The goal is simple: take a familiar project and make one meaningful change without relying on exact instructions. That is where confidence starts to grow.

It is also worth combining the skills you learned into a small original game. For example, you might create a prototype that uses input handling, physics interactions, UI feedback, event-driven communication, and object pooling all at once. The goal is to move from isolated techniques to a complete gameplay loop. Even a tiny game can teach you a lot when its systems need to work together.

As your projects become more ambitious, keep returning to the official Unity documentation. It is one of the best habits you can build, because it helps you move from tutorial-following to independent problem solving.

- [Unity Manual](https://docs.unity3d.com/Manual/index.html) (https://docs.unity3d.com/Manual/index.html)
- [Unity Scripting API](https://docs.unity3d.com/ScriptReference/) (https://docs.unity3d.com/ScriptReference/)

When you read the docs now, you will notice something encouraging: much more of it will make sense than it did at the beginning.

You should also consider joining the wider Unity community. Game development gets easier, and more fun, when you can ask questions, share progress, and see how other developers solve similar problems.

Sharing your work, even unfinished work, is valuable. Feedback helps you improve, and seeing other developers struggle and learn reminds you that progress is rarely linear.

Finally, keep making small projects. Not every idea needs to become a full game. Short experiments are one of the fastest ways to improve, because they let you practice one mechanic, one system, or one design challenge at a time.

To remain current with the shifting landscape of game development, [Zenva Academy](https://academy.zenva.com) (https://academy.zenva.com) offers deep-dive, project-led instruction in Unity. Access to 300+ courses and 40+ Learning Pathways provides a structured environment for those who wish to stay informed on the latest engine features and best practices.

Final Thought

Unity game development is not a finish line, it is a practice. Every prototype teaches you something, every bug sharpens your thinking, and every completed feature makes the next one easier to build.

So keep going. Keep experimenting, keep breaking things and fixing them, and keep making games that are a little more ambitious than the last. You

have already built a strong foundation, and what comes next is where your own voice as a developer begins to show.

OceanofPDF.com

Full Source Code

In this appendix, you will find the complete source code for the Unity projects used throughout the book. This chapter is intended as a practical reference, so you can return to it whenever you need to compare your implementation, debug a problem, or review how each pattern was structured.

The scripts are grouped by pattern, matching the project folders used in the examples from earlier chapters.

Singleton Pattern

This section contains the scripts used for the singleton example. Together, they show both the reusable generic singleton base class and a GameManager that inherits from it.

GameManager.cs

Found in Project/Assets/Singleton Pattern

This script defines a GameManager class that inherits from a generic Singleton base class. It stores the game's score and exposes a method for increasing that score.

```
using UnityEngine;

public class GameManager : Singleton<GameManager>
{
    public int Score { get; private set; }

    public void IncreaseScore (int amount)
    {
        Score += amount;
    }
}
```

Singleton.cs

Found in Project/Assets/Singleton Pattern

This generic singleton base class ensures that only one instance of a component type exists at runtime. In Awake(), it either assigns the shared instance or destroys duplicates, and it also persists the object across scene loads with DontDestroyOnLoad().

```
using UnityEngine;

public class Singleton<T> : MonoBehaviour where T : Component
{
    public static T Instance { get; private set; }

    // Destroy duplicates at startup; keep the first instance
    // alive across scene loads
    public virtual void Awake ()
    {
        if(Instance != null && Instance != this)
        {
            Destroy(gameObject);
        }
        else
        {
            // as T is required because 'this' has type
            Singleton<T>, not T itself
            Instance = this as T;
            DontDestroyOnLoad(gameObject);
        }
    }
}
```

TestScript.cs

Found in Project/Assets/Singleton Pattern

This simple test script demonstrates how another component can access the singleton instance directly. When the scene starts, it increases the score by 1 through GameManager.Instance.

```
using UnityEngine;
```

```

public class TestScript : MonoBehaviour
{
    void Start ()
    {
        GameManager.Instance.IncreaseScore(1);
    }
}

```

Observer Pattern

The following scripts make up the observer pattern example. In this setup, LightManager broadcasts events and each LightController listens for them.

LightController.cs

Found in Project/Assets/Observer Pattern

This script controls the visual state of a light by swapping materials on a MeshRenderer. It subscribes to the manager's on and off events in OnEnable() and unsubscribes in OnDisable().

```

using UnityEngine;

public class LightController : MonoBehaviour
{
    [SerializeField] private MeshRenderer meshRend;
    [SerializeField] private Material offMaterial;
    [SerializeField] private Material onMaterial;

    // Subscribe when the light becomes active
    void OnEnable ()
    {
        LightManager.Instance.OnTurnOn += TurnOn;
        LightManager.Instance.OnTurnOff += TurnOff;
    }

    // Unsubscribe to prevent calls on a disabled or destroyed
    object
    void OnDisable ()
    {
        LightManager.Instance.OnTurnOn -= TurnOn;
        LightManager.Instance.OnTurnOff -= TurnOff;
    }
}

```

```

void TurnOn ()
{
    meshRend.material = onMaterial;
}

void TurnOff ()
{
    meshRend.material = offMaterial;
}
}

```

LightManager.cs

Found in Project/Assets/Observer Pattern

This script acts as the observable object in the pattern. It exposes OnTurnOn and OnTurnOff events, provides singleton-style access through Instance, and invokes the appropriate event when the player presses the 1 or 2 keys.

```

using UnityEngine;
using UnityEngine.Events;

// Run Awake before default order so Instance is ready when
// observers subscribe
[DefaultExecutionOrder(-1)]
public class LightManager : MonoBehaviour
{
    public event UnityAction OnTurnOn;
    public event UnityAction OnTurnOff;

    public static LightManager Instance { get; private set; }

    void Awake ()
    {
        if(Instance != null && Instance != this)
        {
            Destroy(gameObject);
        }
        else
        {
            Instance = this;
        }
    }
}

```

```

void Update()
{
    if(Input.GetKeyDown(KeyCode.Alpha1))
    {
        OnTurnOn?.Invoke();
    }
    else if(Input.GetKeyDown(KeyCode.Alpha2))
    {
        OnTurnOff?.Invoke();
    }
}
}

```

Command Pattern

This section collects the full set of scripts used for the command pattern example. These scripts work together to execute player actions and support undo and redo behavior.

CommandExecutor.cs

Found in Project/Assets/Command Pattern

This script listens for player input, creates command objects, executes them, and stores them in undo and redo stacks. It is the central coordinator for the command system.

```

using UnityEngine;
using System.Collections.Generic;

public class CommandExecutor : MonoBehaviour
{
    [SerializeField] private PlayerMovement playerMovement;

    private Stack<ICommand> undoStack = new Stack<ICommand>();
    private Stack<ICommand> redoStack = new Stack<ICommand>();

    void Update ()
    {
        if(Input.GetKeyDown(KeyCode.UpArrow))
            ExecuteCommand(new MoveCommand(playerMovement,
Vector3.up));
    }
}

```

```

        if(Input.GetKeyDown(KeyCode.DownArrow))
            ExecuteCommand(new MoveCommand(playerMovement,
Vector3.down));

        if(Input.GetKeyDown(KeyCode.LeftArrow))
            ExecuteCommand(new MoveCommand(playerMovement,
Vector3.left));

        if(Input.GetKeyDown(KeyCode.RightArrow))
            ExecuteCommand(new MoveCommand(playerMovement,
Vector3.right));

        if(Input.GetKeyDown(KeyCode.S))
            ExecuteCommand(new ScaleCommand(playerMovement, new
Vector3(0.2f, 0.2f, 0.2f)));
    }

    void ExecuteCommand (ICommand command)
    {
        command.Execute();
        undoStack.Push(command);
        // Discard the redo chain: a new action invalidates
previous undone commands
        redoStack.Clear();
    }

    public void UndoCommand ()
    {
        if(undoStack.Count == 0)
            return;

        ICommand currentCommand = undoStack.Pop();
        // Move to the redo stack so this command can be
replayed if needed
        redoStack.Push(currentCommand);
        currentCommand.Undo();
    }

    public void RedoCommand ()
    {
        if(redoStack.Count == 0)
            return;

        ICommand currentCommand = redoStack.Pop();
        // Return to the undo stack so the replayed command can
be undone again
        undoStack.Push(currentCommand);
    }

```

```

        currentCommand.Execute();
    }
}

```

ICommand.cs

Found in Project/Assets/Command Pattern

This interface defines the shared contract for all commands in the system. Any command that implements ICommand must provide both Execute() and Undo() methods.

```

using UnityEngine;

public interface ICommand
{
    void Execute();
    void Undo();
}

```

MoveCommand.cs

Found in Project/Assets/Command Pattern

This command stores a movement direction and applies it through PlayerMovement. Its Undo() method reverses the action by applying the opposite movement vector.

```

using UnityEngine;

public class MoveCommand : ICommand
{
    private PlayerMovement playerMovement;
    private Vector3 movement;

    public MoveCommand (PlayerMovement playerMovement, Vector3 movement)
    {
        this.playerMovement = playerMovement;
        this.movement = movement;
    }

    public void Execute ()

```

```

    {
        playerMovement.Move(movement);
    }

    public void Undo ()
    {
        // Reverse the move by applying the opposite direction
        playerMovement.Move(-movement);
    }
}

```

PlayerMovement.cs

Found in Project/Assets/Command Pattern

This script performs the actual transform changes requested by the commands. It provides one method for movement and another for scaling.

```

using UnityEngine;

public class PlayerMovement : MonoBehaviour
{
    public void Move (Vector3 movement)
    {
        transform.Translate(movement);
    }

    public void Scale (Vector3 scaleAmount)
    {
        transform.localScale += scaleAmount;
    }
}

```

ScaleCommand.cs

Found in Project/Assets/Command Pattern

This command applies a scale change to the player and can reverse that change through Undo(). It follows the same structure as MoveCommand, which is what makes the command system easy to extend.

```

using UnityEngine;

```

```

public class ScaleCommand : ICommand
{
    private PlayerMovement playerMovement;
    private Vector3 scaleAmount;

    public ScaleCommand (PlayerMovement playerMovement, Vector3
scaleAmount)
    {
        this.playerMovement = playerMovement;
        this.scaleAmount = scaleAmount;
    }

    public void Execute()
    {
        playerMovement.Scale(scaleAmount);
    }

    public void Undo()
    {
        // Reverse the scale by applying the opposite amount
        playerMovement.Scale(-scaleAmount);
    }
}

```

Object Pooling

The final group of scripts covers the object pooling example. In this setup, projectiles are reused instead of being repeatedly instantiated and destroyed.

ObjectPool.cs

Found in Project/Assets/Object Pooling

This script manages a pool of reusable objects. It pre-creates a minimum number of inactive instances, returns an inactive object when requested, and expands the pool automatically if all existing objects are in use.

```

using UnityEngine;
using System.Collections.Generic;

public class ObjectPool : MonoBehaviour
{
    [SerializeField] private GameObject prefab;

```

```

[SerializeField] private float minimumAmount;

private List<GameObject> pool = new List<GameObject>();

void Start ()
{
    for(int i = 0; i < minimumAmount; i++)
    {
        CreateNewObject();
    }
}

GameObject CreateNewObject ()
{
    GameObject obj = Instantiate(prefab);
    pool.Add(obj);
    obj.SetActive(false);

    return obj;
}

public GameObject GetObject ()
{
    GameObject obj = pool.Find(x => x.activeSelf == false);

    // Grow the pool automatically if all objects are
currently active
    if(obj == null)
        obj = CreateNewObject();

    obj.SetActive(true);

    return obj;
}
}

```

Projectile.cs

Found in Project/Assets/Object Pooling

This script handles projectile movement and lifetime. When initialized, it applies velocity to the projectile and schedules a despawn by deactivating the object after a delay.

```

using UnityEngine;

public class Projectile : MonoBehaviour
{
    [SerializeField] private float speed;
    [SerializeField] private float lifetime;

    private Rigidbody rig;

    void Awake ()
    {
        rig = GetComponent<Rigidbody>();
    }

    public void Initialize (Vector3 direction)
    {
        rig.linearVelocity = direction * speed;

        //Destroy(gameObject, lifetime);
        // nameof avoids a fragile string literal if Despawn is
ever renamed
        Invoke(nameof(Despawn), lifetime);
    }

    void Despawn ()
    {
        gameObject.SetActive(false);
    }
}

```

Shooter.cs

Found in Project/Assets/Object Pooling

This script handles player movement and firing. Instead of instantiating a new projectile every time, it requests one from the object pool, positions it, and initializes it with a random firing direction.

```

using UnityEngine;

public class Shooter : MonoBehaviour
{
    [SerializeField] private GameObject projectilePrefab; //
Not being used anymore
    [SerializeField] private float shootRate;

```

```

private float lastShootTime;

[SerializeField] private ObjectPool projectilePool;

void Update ()
{
    // Movement
    Vector2 moveInput = new
Vector2(Input.GetAxisRaw("Horizontal"),
Input.GetAxisRaw("Vertical")).normalized;
    transform.Translate(moveInput * 2.0f * Time.deltaTime);

    // Shooting
    if(Input.GetKey(KeyCode.Space))
    {
        if(Time.time - lastShootTime > shootRate)
        {
            Shoot();
        }
    }
}

void Shoot ()
{
    lastShootTime = Time.time;

    //GameObject proj = Instantiate(projectilePrefab,
transform.position, Quaternion.identity);
    GameObject proj = projectilePool.GetObject();

    // Reposition before use: pooled objects retain the
transform from their last active state
    proj.transform.position = transform.position;

    Vector3 shootDirection = new
Vector3(Random.Range(-1.0f, 1.0f), Random.Range(-1.0f, 1.0f),
0).normalized;
    proj.GetComponent<Projectile>
().Initialize(shootDirection);
}
}

```